



ProSeller — Regras de Negócio & Core

Contrato vivo · verificado com passada adversarial · 03/07/2026

ProSeller — Regras de Negócio

ProSeller — Regras de Negócio & Core

Bloco 1 — Regras de Ouro do Core

1. Pedido & Emissão ao ERP (Tiny)
2. Cliente
3. Fiscal — Simples Nacional & Natureza
4. Condição de Pagamento
5. Comissão
6. Logística & Rastreio (SSW)
7. Permissões, Usuários & Acesso
8. Produtos, Listas de Preço, Relatórios & Conta Corrente
9. Operação & Deploy (aprendido no susto)
10. Armadilhas conhecidas (bugs que já corrigimos — não deixar voltar)

Bloco 2 — Regras de Negócio Detalhadas (por domínio)

Pedido & Emissão ao ERP

- Regras de negócio
- Dúvidas em aberto

Cliente

- Regras de negócio
- Dúvidas em aberto

Fiscal: Simples & Natureza

- Regras de negócio
- Dúvidas em aberto

Condições de Pagamento

- Regras de negócio
- Dúvidas em aberto

Comissão

- Regras de negócio
- Dúvidas em aberto

Logística & Rastreio SSW

- Regras de negócio
- Dúvidas em aberto

Permissões & Acesso

- Regras de negócio
- Dúvidas em aberto

Produtos, Listas de Preço & Comissionamento

- Regras de negócio
- Dúvidas em aberto

Relatórios

Regras de negócio
Dúvidas em aberto
Conta Corrente
Regras de negócio
Dúvidas em aberto

ProSeller — Regras de Negócio & Core

Documento vivo. Bloco 1 = regras de ouro (resumo executivo). Bloco 2 = regras detalhadas por domínio, cada uma com **Dúvidas em aberto** (marcadas para resolução via app/Playwright ou decisão do cliente). Verificado com passada adversarial multi-agente.

Bloco 1 — Regras de Ouro do Core

1. Pedido & Emissão ao ERP (Tiny)

1. **Sucesso só com confirmação do Tiny.** O pedido só é “enviado ao ERP” quando o Tiny devolve o ID/número dele. Sem isso = falha, fica pra reenviar. *Regressão = mostrar “enviado com sucesso” sem ID do Tiny.*
2. **Nota só com regime confirmado.** Se a consulta do Simples falhar (e a empresa tem natureza dupla), o envio é **bloqueado**. *Regressão = emitir mesmo assim com natureza chutada.*
3. **Natureza correta por empresa.** A natureza enviada ao Tiny é a mapeada para aquela empresa; o regime Simples pode trocá-la (mapeamento duplo). *Regressão = enviar sem mapeamento → Tiny usa a operação padrão errada.*
4. **Pré-requisitos de emissão:** vendedor com `idtiny`, cliente com CPF/CNPJ válido (11 ou 14 dígitos), empresa com chave de API, e pedido com pelo menos 1 item. Faltando qualquer um, não emite.
5. **Rascunho não vai ao ERP.** Pedido em Rascunho não pode ser enviado.
6. **Número muda no envio.** O número interno (`PV-2025-XXXX`) é gerado na tela; após o envio vira o número do Tiny. *Não usar o PV como chave de busca depois — ele “some”.*
7. **Bonificação não gera comissão** (mas é pedido válido).

2. Cliente

8. **Salvar nunca apaga o que você não mexeu** — endereço, contato, observações, grupo, condições. *Regressão = editar um campo e outro sumir.*
9. **Exclusão é arquivar (soft delete).** Some das listas/contagens/relatórios, mas nada é apagado de vez. Só backoffice exclui.
10. **Nome obrigatório** (mínimo 2 caracteres; só espaços = vazio).
11. **Grupo/Rede aparece no detalhe** do cliente (não só na lista).
12. **Condições de pagamento do cliente:** ao editar, a lista é substituída por completo (não vai somando).

3. Fiscal — Simples Nacional & Natureza

13. **Regime é consultado e guardado** (ReceitaWS), com data da consulta. *“Não consultado” ≠ “não optante”* — nunca tratar vazio como “não é do Simples”.
14. **Revalidação no envio ao ERP** para empresas de natureza dupla; se falhar, bloqueia (ver regra 2).
15. **Cliente PJ tem o Simples exibido no cadastro** (Sim / Não / Não consultado / Indisponível).

4. Condição de Pagamento

- 16. **O nome mostra TODAS as parcelas** (ex.: “10/15/20 dias”), não só a última.
- 17. **O faturamento usa o dado real das parcelas** (o intervalo), **nunca o nome** nem um prazo único.
Regressão = calcular vencimentos a partir do nome.
- 18. **Quantidade de parcelas = tamanho do intervalo** (sempre bate).

5. Comissão

- 19. **Pedido excluído nunca gera comissão** nem entra em relatório de comissão. *Regressão = comissão de pedido que foi excluído.*
- 20. **Comissão só de venda real** (não cancelada, não excluída, não bonificação).
- 21. **Regras de comissão:** fixa (percentual do vendedor) ou por faixa de desconto (quanto maior o desconto, menor a comissão).
- 22. **Só backoffice edita/exclui comissão.**

6. Logística & Rastreio (SSW)

- 23. **Entrega é definitiva (“sticky”).** Um frete entregue/devolvido não volta pra “em trânsito” por causa de evento administrativo posterior (ex.: “anexado comprovante”). *Regressão = frete entregue voltando pra “em trânsito”.*
- 24. **O status considera o histórico todo**, não só o último evento.
- 25. **Frete já entregue não é mais consultado** no SSW (status terminal).
- 26. **Atualização automática de rastreio a cada 1h** (+ botões manuais no Kanban e no detalhe do frete).
- 27. **Romaneio = notas em separação ainda não romaneadas** (filtro por transportador opcional).

7. Permissões, Usuários & Acesso

- 28. **Dois tipos:** backoffice (vê tudo) e vendedor (vê só os próprios clientes/pedidos/comissões).
- 29. **As permissões valem também para o backoffice** (páginas e ações).
- 30. **Ações sensíveis (excluir cliente, editar comissão, fechar período)** são restritas a backoffice.

8. Produtos, Listas de Preço, Relatórios & Conta Corrente

- 31. **Preço e comissão saem da lista de preço do cliente** + faixa de desconto aplicada.
- 32. **Relatórios (Curva ABC, Mix, comissão) ignoram excluídos e respeitam o vendedor logado.**
- 33. **Conta corrente:** compromissos e pagamentos por cliente; saldo reflete o que foi lançado vs pago.

9. Operação & Deploy (aprendido no susto)

- 34. **Produção = branch `main`**. A Netlify publica a `main`. **NÃO** é a `master` (baseline antiga). PR de front sempre na `main`.
- 35. **Cuidado com clone raso (shallow).** Ele dá *falsa* impressão de divergência (“push não funciona”, “main atrás”). O GitHub geralmente está íntegro. *Nunca force-push*; para subir 1 commit com segurança, usar worktree isolado sobre `origin/main`.
- 36. **Backend Supabase é compartilhado** — todo deploy de edge/migration afeta a produção real, independente da branch.
- 37. **Migration em produção = confirmação humana**, com plano de rollback. Nunca aplicar no susto.
- 38. **Backup antes de mexer em edge/RPC** (guardar a versão atual para rollback em 1 comando).
- 39. **Commitar antes de fazer deploy**; deploy sai da `main`.
- 40. **Mexeu em `src/` (front)? Suba a versão (`systemVersion`)** e registre no changelog.

41. **Edge nova respeita o padrão:** autenticada (verify_jwt) por padrão; pública (webhook/cron) só com proteção (secret).
42. **Validar em transação revertível (dry-run) antes de aplicar** mudança sensível em prod — prova o antes/depois sem gravar nada.

10. Armadilhas conhecidas (bugs que já corrigimos — não deixar voltar)

Já aconteceu	A regra que protege
Salvar cliente apagava a observação de contato (~89 clientes)	Regra 8 — salvar nunca apaga campo não editado
Emissão mostrava “sucesso” sem ir ao Tiny	Regra 1 — sucesso só com ID do Tiny
Comissão de pedido excluído (Donato)	Regra 19 — excluído não comissiona
Frete entregue preso em “em trânsito”	Regra 23 — entrega é definitiva
Nome de condição parcelada mostrava só a última parcela	Regra 16 — nome com todas as parcelas
Grupo/Rede não aparecia no detalhe do cliente	Regra 11 — grupo aparece no detalhe
Regime Simples sumia no cadastro	Regra 15 — Simples exibido
Pedido “sumia” após envio	Regra 6 — o número muda no envio (não sumiu, virou nº Tiny)
Push/branch confusos (main vs master / shallow)	Regras 34–35

Detalhe técnico completo (151 regras, com evidência arquivo:linha, contraexemplos e checklist de PR): ver [docs/specs/system-contract/](#) e [SYSTEM-CONTRACT-COMPLETO.pdf](#).

Bloco 2 — Regras de Negócio Detalhadas (por domínio)

Pedido & Emissão ao ERP

Domínio: envio de pedido de venda ao ERP Tiny via edge function `tiny-enviar-pedido-venda-v1`. Regras derivadas do código atual (edge function + migrations 082/143 + `SalesPage.tsx`), reconciliadas com verificação adversarial. Onde o contrato técnico divergia da implementação, prevalece o **código**.

Regras de negócio

1. **Sucesso no envio ao ERP é confirmado apenas pela ID do Tiny na resposta.** O envio só é considerado bem-sucedido quando o Tiny retorna `registro.id`. Sem ID, o pedido não foi efetivamente registrado no ERP. *Por quê:* mostrar “sucesso” sem ID leva o usuário a ignorar erros reais — o pedido não existe no ERP. *Regressão:* aceitar sucesso sem ID retornado; ou reenviar pedido que já tem `id_tiny` preenchido (ver R11). *Implementação:* `if (!registro.id)` lança erro; `id_tiny` só é gravado após confirmação (edge function, linhas ~564-572).

2. **Natureza de operação é obrigatória e mapeada por empresa antes do envio.** O envio busca o mapeamento em `tiny_empresa_natureza_operacao` (empresa + natureza) e lança erro se o mapeamento estiver ausente. A natureza (`tiny_valor`) só entra no payload do Tiny quando existir valor não vazio. *Por quê:* sem mapeamento, o Tiny usaria sua natureza padrão em vez da pretendida — a NF sairia mal classificada no ERP. *Regressão:* enviar sem validar a existência do mapeamento; enviar com `tiny_valor` vazio/whitespace e deixar o Tiny cair no default silenciosamente (ver N4). *Nota (ajuste):* a validação garante que o mapeamento existe e é NOT NULL, mas **não valida** que `tiny_valor` está efetivamente preenchido/é válido para o Tiny. Não existe dual-mapping (`tiny_valor_simples` não existe na tabela — ver Dúvidas D1/D2).
3. **Pré-requisitos para envio: vendedor com `idtiny`, cliente com CPF/CNPJ de 11 ou 14 dígitos, empresa com chave API Tiny.** O envio faz early-fail se qualquer um faltar. *Por quê:* qualquer campo faltando = rejeição pelo Tiny ou erro no envio; validar cedo economiza tentativas. *Regressão:* enviar sem `vendedor.idtiny`, sem `chave_api` da empresa, ou com CPF/CNPJ vazio/tamanho inválido. *Nota (ajuste):* a validação de CPF/CNPJ checka **apenas o número de dígitos** (11 ou 14 após remover não-dígitos), **não** valida dígito verificador (ver N5).
4. **Pedido em status “Rascunho” não é enviável ao ERP.** Rascunho = trabalho em progresso. *Por quê:* enviar sem confirmação explícita pode gerar pedidos indesejados no ERP. *Regressão:* enviar Rascunho ao Tiny sem bloqueio. *Nota (ajuste — bloqueio frágil):* o bloqueio existe **apenas no frontend** (`SalesPage.tsx`, ~linhas 998-1001). A edge function **não** valida `status` (não busca a coluna no SELECT inicial), então uma chamada direta à function envia um Rascunho. Débito técnico: mover a validação para o backend.
5. **Número local do pedido (`PV-YYYY-XXXX`) é substituído pelo número do Tiny após envio bem-sucedido.** O número local é temporário (gerado em `SaleFormPage.tsx`); após sucesso, `numero_pedido` recebe o número do Tiny. *Por quê:* após o envio, o identificador verdadeiro é o do Tiny; confundir os dois causa busca errada na integração. *Regressão:* usar o número local como chave no ERP ou para reenvios; descartar o número do Tiny. *Nota:* o Tiny **pode não retornar** número — há fallback `registro.numero || pedido.numero_pedido || null` (ver D9/N2).
6. **Pedido deve ter pelo menos 1 item antes de enviar ao ERP.** *Por quê:* pedido vazio é inválido; o Tiny rejeita e o backend lança erro. *Regressão:* permitir envio de pedido sem itens; lista vazia passar pela validação. *Implementação:* `if (!itensDb || itensDb.length === 0)` lança erro.
7. **Bonificação (natureza de operação) é um pedido válido, mas não gera comissão.** *Por quê:* bonificação é presente, não venda — não deve remunerar o vendedor. *Regressão:* pedido de bonificação gerar comissão por ser confundido com venda normal. *Nota (ajuste — falha de case-sensitivity):* a verificação é **case-sensitive e exata** (`natureza_operacao = 'Bonificação'`, sem `LOWER / TRIM`, migrations 082 linha 49 e 143 linha 57). Variações como `'bonificacao'`, `'BONIFICAÇÃO'` ou `'Bonificacao'` **contornam a regra e geram comissão indevidamente**. Débito técnico: normalizar a comparação.
8. **Pedido deletado (soft-delete, `deleted_at` preenchido) não pode ser enviado ao ERP.** *Por quê:* deletado = fora de operação; reenviar cria duplicação e inconsistência no ERP. *Regressão:* aceitar pedido com `deleted_at` para reenvio. *Implementação:* `if (!pedido || pedido.deleted_at)` lança NOT FOUND.
9. **Número de parcelas = tamanho do array `intervalo_parcela` da condição de pagamento.** As parcelas são geradas a partir de `intervalos.length`. *Por quê:* integridade de dados — se os números divergem, os vencimentos ficam errados. *Regressão:* permitir mismatch entre número de parcelas e intervalos. *Nota (ajuste):* **não há validação cruzada** de tamanho. A integridade depende de `intervalo_parcela` estar correto no banco; dados corrompidos geram mismatch silencioso. Intervalos todos inválidos caem em fallback `[0]` (à vista — ver N9).

10. **Envio deve ser idempotente: pedido que já tem `id_tiny` preenchido não deve ser reenviado.**
Por quê: reenviar duplica o pedido no Tiny (ou gera erro do Tiny) e cria inconsistência. *Regressão:* a edge function **não valida idempotência** — o SELECT inicial não inclui `id_tiny`, então a function **aceita reenvio** de pedido já emitido. Débito técnico confirmado (D8): incluir `id_tiny` no SELECT e bloquear/curto-circuitar quando já preenchido.
11. **Em caso de falha no envio ao Tiny, `id_tiny` não é limpo.** O `UPDATE` de `id_tiny / numero_pedido` ocorre **somente após** sucesso confirmado; em erro, a exceção é capturada e nada é limpo. *Por quê:* previne sobrescrever com estado parcial em uma tentativa que falhou. *Regressão/risco:* se havia um `id_tiny` obsoleto anterior, ele permanece **dangling**. Combinado com a ausência de checagem de idempotência (R10), isso pode mascarar o estado real do pedido.

Regra removida

- ~“Regime Simples Nacional bloqueia emissão ao ERP se consulta falhar e empresa tem dual-mapping”~ — **REFUTADA / NÃO IMPLEMENTADA.** Não existe dual-mapping nem consulta ao ReceitaWS no fluxo de envio. A tabela `tiny_empresa_natureza_operacao` tem apenas `tiny_valor` (migration 085), sem `tiny_valor_simples`. A edge function não importa/chama nenhum lookup de Simples Nacional, e as colunas `optante_simples_nacional`, `tiny_natureza_enviada`, `tiny_optback_used / tiny_fallback_used` existem no schema mas **nunca são preenchidas nem consultadas**. O contrato técnico está fora de sincronismo com o código. Não há risco de “fallback NULL silencioso” porque a lógica simplesmente não roda — mas também **não há proteção tributária** para empresas que saem do Simples.

Dúvidas em aberto

- **D1/D2 — Dual-mapping de natureza para Simples vs não-Simples.** [RESPONDIDA] Não existe. A coluna `tiny_valor_simples` **não existe** na tabela (`schema_baseline.sql` / migration 085 tem só `tiny_valor`). Não há função `resolveNaturezaTiny` nem lógica condicional por regime. O contrato descreve uma feature não implementada. *Resolvido via:* código.
- **D3/D4/D10 — Consulta ReceitaWS (Simples Nacional) e comportamento com `optante_simples_nacional` NULL.** [RESPONDIDA] Nunca é feita no fluxo de envio ao Tiny. `optante_simples_nacional` permanece NULL e não bloqueia nada (fallback silencioso porque a lógica não roda). Sem revalidação de Simples no envio. *Resolvido via:* código.
- **D5 — Case-sensitivity de “Bonificação”.** [RESPONDIDA] Confirmado case-sensitive e exato (sem `LOWER / TRIM`). Variações de grafia geram comissão indevida (ver R7). Recomenda-se validação em Playwright para confirmar em ambiente real quais grafias existem em produção. *Resolver via:* Playwright (confirmação em prod) — comportamento de código já confirmado.
- **D6 — Fluxo de status de Rascunho até “enviado/faturado”.** [RESPONDIDA PARCIALMENTE] Criação default = `Rascunho` (`pedido-venda-v2`). Frontend bloqueia envio de Rascunho. **Após envio bem-sucedido ao Tiny, nenhuma alteração automática de status ocorre** — o status permanece como estava. Não há webhook conhecido que promova para “Faturado”/“Em aberto”. *Aberto:* confirmar com o cliente/produto se a ausência de transição de status pós-envio é intencional, ou se falta implementar (webhook do Tiny ou update na edge function). *Resolver via:* cliente.
- **D7 — `id_tiny` em caso de falha.** [RESPONDIDA] Mantém valor anterior (não é limpo); ver R11. Risco de valor dangling. *Resolvido via:* código.
- **D8 — Reenvio de pedido já com `id_tiny`.** [RESPONDIDA] Não há bloqueio de idempotência; a function aceita reenvio. Ver R10 (débito técnico). *Resolvido via:* código.
- **D9/N2 — Número do Tiny pode vir nulo/vazio.** [RESPONDIDA PARCIALMENTE] Sim, pode. Fallback: `registro.numero || pedido.numero_pedido || null`. *Aberto:* confirmar se `numero_pedido` aceita NULL no banco e o impacto em integrações externas quando fica NULL.

Resolver via: código (schema/constraint) + cliente (impacto de integração).

- **N6 — Sucesso no Tiny mas falha no UPDATE local.** [ABERTA] Se o Tiny cria o pedido mas o `UPDATE` local de `id_tiny` falha, o pedido fica com `id_tiny = NULL` no banco enquanto **já existe no Tiny**. Não há evidência de log/alerta/recuperação. Combinado com R10 (sem idempotência), o próximo reenvio duplicaria no Tiny. *Resolver via:* código (adicionar log/alerta + reconciliação) — decisão de produto sobre estratégia de recuperação.
- **N1 — Chave API expirada/inválida.** [ABERTA] O código só verifica que `chave_api` não está vazia; não valida o token antes do envio. O error handling é genérico e não diferencia falha de autenticação de outros erros do Tiny. *Resolver via:* código (melhorar tratamento e mensagem).
- **N7 — Autorização além do JWT.** [ABERTA] Vendedor é restrito a enviar seus próprios pedidos, mas não há evidência clara de validação de quem no backoffice pode enviar pedidos de terceiros além do JWT básico. *Resolver via:* código + cliente (definição de política de permissão).
- **N8 — Valores negativos.** [ABERTA] `valor_total` / `valor_final` são calculados sem validar que preços/itens são positivos; item com `valor_unitario` negativo pode gerar parcela negativa. *Resolver via:* código (validação) — confirmar com cliente se negativos são um caso legítimo.

Cliente

Regras de negócio

1. **Nome (razão social) obrigatório com no mínimo 2 caracteres úteis.** O nome não pode ser vazio, ter menos de 2 caracteres nem conter apenas espaços em branco (a validação desconsidera espaços). A RPC deve rejeitar antes de persistir.
 - *Por quê:* O nome é o identificador principal do cliente; nomes vazios ou irrisórios causam confusão e violam integridade básica de dados.
 - *Regressão:* Aceitar nome vazio, com menos de 2 caracteres ou só espaços; salvar no banco sem validação na RPC.
2. **Exclusão de cliente é soft delete e restrita a backoffice.** Excluir marca `deleted_at` e define `situação = 'Excluído'`; não há hard delete. Vendedores não podem excluir; apenas usuários backoffice executam a exclusão.
 - *Por quê:* Preserva auditoria e histórico; impede que vendedor apague dados de clientes de terceiros; operações críticas ficam com o backoffice.
 - *Regressão:* Vendedor consegue excluir; uso de hard delete; exclusão sem atualizar `situação`; backoffice sem conseguir excluir.
3. **Optante Simples Nacional é cacheado com timestamp; NULL significa “desconhecido”.** O valor é buscado na ReceitaWS (CONSOPT) e armazenado em `optante_simples_nacional` junto de `optante_simples_nacional_consultado_em`. `NULL` NÃO é `false` — indica nunca consultado / pessoa física / resultado inconclusivo. Na revalidação ao enviar para o Tiny, se a consulta falhar em empresa dual-mapped, a emissão é bloqueada. O lookup é best-effort e ocorre FORA do `create_cliente_v2` (a RPC de criação não popula esses campos).
 - *Por quê:* Garante regime fiscal correto antes da emissão de nota; `NULL` desconhecido não pode ser tratado como negativo.
 - *Regressão:* Valor não cacheado ou timestamp perdido; lookup nunca disparado; `NULL` tratado como `false`; emissão prossegue com regime desconhecido em empresa dual-mapped.
4. **Condições de pagamento: substituição atômica all-or-nothing.** Quando o array de IDs é enviado, o UPDATE faz `DELETE` de todas as condições existentes e depois `INSERT` das novas. Se o array não é enviado (`NULL`), as condições não são tocadas. Cliente pode existir sem nenhuma condição

(array opcional). No INSERT de IDs inexistentes há `ON CONFLICT DO NOTHING` — a linha é ignorada silenciosamente (ver Dúvidas).

- *Por quê:* Evita estado inconsistente (condições órfãs); o array enviado é a fonte da verdade completa.
- *Regressão:* Novas condições somadas às existentes em vez de substituir; updates parciais deixando linhas órfãs; ignorar o `DELETE+INSERT` quando o array é enviado.

5. **`ref_tipo_pessoa_id_FK` protegido por FOREIGN KEY e validação explícita; NULL permitido.**

`update_cliente_v2` valida explicitamente o ID (`IF EXISTS`) além da constraint FK. `NULL` é aceito (tipo indeterminado).

- *Por quê:* O tipo de pessoa (Física/Jurídica) define natureza fiscal e validações de documento; a FK garante integridade.
- *Regressão:* IDs inválidos aceitos; remoção da FK; tipo armazenado como string; remoção da validação no UPDATE.

6. **Empresa de faturamento é nullable e preservada em update parcial.** No UPDATE, `CASE WHEN`

preserva o valor atual quando o campo não é enviado. A FK usa `ON DELETE SET NULL`. Diferente de `ref_tipo_pessoa_id`, NÃO há validação explícita `IF EXISTS` — a integridade depende apenas da constraint FK, e um ID inválido gera erro do banco propagado ao frontend (design choice, não bug).

- *Por quê:* Nem todo cliente tem empresa de faturamento customizada; preservar valor existente é princípio de segurança em updates parciais.
- *Regressão:* Campo vira NOT NULL; FK removida ou com comportamento alterado; valor sobrescrito para NULL em update parcial.

7. **`desconto_financeiro` e `pedido_minimo` : default 0 no INSERT, preserva no UPDATE.** No

INSERT, frontend converte e SQL usa `COALESCE` garantindo 0. No UPDATE, `COALESCE` com o campo atual preserva o valor quando não enviado.

- *Por quê:* Evita NULL em campos numéricos; INSERT sempre resulta em 0, UPDATE mantém integridade em payload parcial.
- *Regressão:* NULL armazenado em vez de 0; remoção do `COALESCE` no INSERT; UPDATE usando NULL direto; comparações numéricas quebrando.

8. **`status_aprovacao` limitado por CHECK a 'aprovado' | 'pendente' | 'rejeitado'.** A constraint

CHECK está corretamente imposta. **Débito conhecido (bug em produção):** a lógica de atribuição no CREATE deveria ser backoffice→'aprovado' e vendedor→'pendente', mas desde a migration 114/116 ambas as branches atribuem SEMPRE 'aprovado' (migration 007 original estava correta). A regra de workflow está violada no CREATE; a constraint em si continua válida.

- *Por quê:* Controle de workflow de aprovação — vendedor deveria criar como pendente (requer análise), backoffice como aprovado.
- *Regressão:* Inserir outros valores; remover a constraint CHECK; transições que não validam o enum. (A correção pendente é fazer o CREATE atribuir 'pendente' para vendedor.)

9. **UPSERT de `cliente_contato` e `cliente_endereço` preserva dados existentes via**

`COALESCE(NULLIF(novo), tabela.campo)`. O fallback usa a coluna da tabela, NÃO `EXCLUDED` — updates parciais (campo não reenviado) não apagam dados armazenados.

- *Por quê:* `EXCLUDED` aponta para o valor vazio proposto, não ao valor armazenado; usar `EXCLUDED` apagaria dados em payload parcial.
- *Regressão:* Fallback com `EXCLUDED`; campos apagados quando NULL é enviado. (A migration 131 reintroduziu esse bug; corrigido na migration 140.)

10. **Contato e endereço são singleton por cliente (PK = cliente_id), atualizados na mesma RPC.**

Cada cliente tem no máximo um `cliente_contato` e um `cliente_endereço`, criados/atualizados por UPSERT na mesma transação. Se nenhum campo desses é fornecido no payload, as tabelas auxiliares não são tocadas (sem DELETE).

- *Por quê:* Singularidade garantida por PK; transação atômica; enviar payload sem esses campos não deve apagar dados.

- *Regressão*: Cliente com múltiplos contatos/endereços; `DELETE` chamado quando campos são NULL; tabelas auxiliares fora da transação; endpoints separados.
11. **Empresa de faturamento e grupo: dual mapping `grupo_id` (UUID FK) + `grupo_rede` (TEXT legado).** O UPDATE prioriza `grupo_id` via `CASE WHEN p_grupo_id IS NOT NULL`. A edge function (POST/PUT/CREATE) aplica o padrão de nulificação `p_grupo_rede: grupoId ? null : (body.grupoRede ?? ...)`, então quando o `grupoId` é resolvido para UUID, `grupo_rede` vira NULL. `get_cliente_completo_v2` retorna `grupo_rede_nome` via JOIN. **Ressalva:** a RPC de CREATE (migration 114/116) NÃO aplica `CASE WHEN` para anular `grupo_rede` — se um payload legado enviar ambos os campos, ambos são inseridos (inconsistência com o UPDATE); na prática o frontend sempre resolve `grupoId` e envia `grupo_rede=null`.
- *Por quê*: Compatibilidade com sistema legado; UUID é a fonte da verdade, texto é fallback de display.
 - *Regressão*: `grupo_id` removido ou nunca populado; `grupo_rede` tratado como primário; `CASE WHEN` virar `COALESCE` (apagaria `grupo_id`); GET sem o JOIN deixando o campo vazio.
12. **`segmento_id` é BIGINT FK nullable para `segmento_cliente_id`; coexiste com `tipo_segmento` (TEXT legado).** Cliente pode não ter segmento (NULL). `get_cliente_completo_v2` inclui `segmento_nome` via LEFT JOIN. São campos DISTINTOS: `tipo_segmento` é legado TEXT (derivado do id como string na edge function), `segmento_id` é a FK moderna; ambos podem estar preenchidos simultaneamente e não há mutex entre eles.
- *Por quê*: Segmento é atributo opcional; LEFT JOIN permite cliente sem segmento aparecer normalmente (`segmento_nome=NULL`).
 - *Regressão*: `segmento_id` vira NOT NULL; FK torna segmento obrigatório; `segmento_nome` fora do GET; segmento armazenado só como texto.
13. **Mappers usam cadeias de nullish coalescing (??) para tolerar variações de schema.** `mapClienteCompleto` e `mapClienteListItem` encadeiam campo legado + novo + variações snake/camelCase (ex.: `grupo_rede_nome ?? grupo_rede ?? grupoRede`).
- *Por quê*: Compatibilidade retrógrada; suporta variações de schema; previne crash/campo vazio quando o nome legado não vem da RPC.
 - *Regressão*: Cadeias removidas; apenas um nome checado; variações não antecipadas (incidente histórico com `grupo_rede_nome`).
14. **Auditoria e controle de acesso por propriedade: `criado_por` e `atualizado_por`.** Clientes rastreiam criador e último atualizador. `update_cliente_v2` valida autorização: vendedor só edita clientes que criou E apenas enquanto `status_aprovacao = 'aprovado'`; se rejeitado, fica bloqueado. Backoffice edita qualquer cliente.
- *Por quê*: Auditoria de mudanças; controle de acesso baseado em propriedade para vendedor; backoffice é administrador.
 - *Regressão*: Vendedor editando clientes de terceiros; UPDATE sem validar `criado_por`; `atualizado_por` não registrado.
15. **`requisitos_logisticos` (JSONB) é substituído all-or-nothing via flag `p_set_requisitos_logisticos`.** Se a flag é `false`, o campo não é tocado (`c.requisitos_logisticos`); se `true`, é REPLACE completo (não há merge parcial). A edge function passa `hasValue`, então campo ausente no body é preservado.
- *Por quê*: Preservação de dados em payload parcial + fonte-da-verdade completa quando o campo é enviado.
 - *Regressão*: Merge parcial silencioso; campo apagado quando não enviado; flag ignorada.

Dúvidas em aberto

1. **Cliente pode ser criado sem nenhuma condição de pagamento — o negócio quer exigir ao menos uma?** [RESPONDIDA — código] Sim, é permitido criar sem condições: o INSERT só ocorre `IF p_condicoes_pagamento_ids IS NOT NULL AND array_length > 0`. Falta confirmar com o **cliente** se a regra de negócio deveria exigir pelo menos uma condição.
2. **Vendedor fica bloqueado de editar o próprio cliente quando backoffice o rejeita (`status='rejeitado'`) — isso é intencional?** [RESPONDIDA — código] Sim, o bloqueio é por design: `update_cliente_v2` barra vendedor quando `status_aprovacao != 'aprovado'`. Confirmar com o **cliente** se esse é o fluxo desejado.
3. **`requisitos_logisticos` deveria permitir merge parcial de chaves internas em vez de replace total?** [RESPONDIDA — código] Hoje é replace all-or-nothing (sem merge). Confirmar com o **cliente** se o negócio espera edição parcial de subcampos do JSONB.
4. **Bug do `status_aprovacao` no CREATE (sempre 'aprovado') — corrigir para atribuir 'pendente' a vendedor?** [RESPONDIDA — código] Confirmado como bug em produção (migration 114/116; a 007 era correta). Continua sem correção nas migrations posteriores. Ação pendente: reintroduzir `vendedor → 'pendente'` no CREATE.
5. **Referências órfãs em `criado_por` / `atualizado_por` e `vendedoresatribuidos`.** [RESPONDIDA — código, sem FK] Não há constraint FK validando esses UUIDs contra a tabela de usuários; `vendedoresatribuidos` é array simples de UUIDs (`COALESCE(p_vendedoresatribuidos, c.vendedoresatribuidos)`), sem `IF EXISTS`. Se um usuário é deletado depois, a referência fica órfã. Abrir com **cliente/equipe** se é necessário mecanismo de validação/cleanup.
6. **IDs inválidos em `condicoes_pagamento_ids` são engolidos silenciosamente (`ON CONFLICT DO NOTHING`).** [RESPONDIDA — código] O cliente é criado sem as condições pretendidas, sem erro nem alerta. Decidir com **cliente/equipe** se deve validar os IDs antes do INSERT e reportar falha.
7. **Fallback para `tipo_segmento` (legado) quando `segmento_id` é NULL.** [RESPONDIDA — código] Se `tipo_segmento` está preenchido e `segmento_id` é NULL, o GET (que retorna `segmento_nome` via JOIN em `segmento_id`) ignora o campo legado. Confirmar com **cliente/equipe** se é preciso fallback de display para `tipo_segmento`.
8. **UPDATE não anula `grupo_id` quando só `grupo_rede` é enviado.** [RESPONDIDA — código] `update_cliente_v2` usa `CASE WHEN p_grupo_id IS NOT NULL` e não trata `p_grupo_rede`; se o frontend enviar `grupoRede` sem `grupoId` resolvido, o `grupo_id` anterior é mantido. Verificar com **equipe** se o comportamento esperado é anular `grupo_id` nesse caso.
9. **Isolamento de dados / RLS.** [RESPONDIDA — código] Não há RLS de isolamento por empresa/região no nível de tabela; a autorização é funcional dentro das RPCs (via `p_requesting_user_id`). Backoffice enxerga e edita TODOS os clientes; a mesma política vale em dev/staging/prod. Consta como débito de segurança conhecido (políticas `allow_all` ainda neutralizam RLS em prod) — acompanhar com **equipe de segurança**.

Fiscal: Simples & Natureza

Aviso de status (verificação adversarial): A grande maioria das regras propostas para este domínio **não está implementada no código atual** do repositório. Elas aparecem em commits do histórico Git (V1.67/V1.68, ~2026-06-25) e no *system-contract*, mas as funções, tabelas e lógica descritas **não existem** no working tree atual. Este documento registra apenas o que é verificável no código, e move o restante para “Dúvidas em aberto”.

Em resumo, hoje: **não há consulta à ReceitaWS para regime Simples Nacional, não há revalidação no envio de pedido, não há seleção entre `tiny_valor` e `tiny_valor_simples`, e não há bloqueio D3 / `REGIME_LOOKUP_FAILED`.**

Regras de negócio

1. **Tipo de pessoa é derivado, nunca consultado externamente.** O `tipoPessoa` em `mapClienteCompleto` (`clientes-v2/index.ts`) vem de (1) `tipo_pessoa_nome` retornado pela RPC `get_cliente_completo_v2`, ou como fallback (2) do comprimento do documento: 11 dígitos → Pessoa Física, 14 dígitos → Pessoa Jurídica. Nenhuma chamada externa (ReceitaWS) é feita para confirmar PF/PJ. *Por quê:* A distinção PF/PJ já está determinada pelo próprio documento e pela classificação cadastrada; consultar API externa só para isso adicionaria latência e dependência sem ganho. *Regressão:* Passar a chamar ReceitaWS (ou outra API) para decidir PF/PJ, ou inverter a regra de comprimento (11↔14), causando classificação incorreta de todos os clientes sem `tipo_pessoa_nome`.
2. **ReceitaWS é usada apenas na criação de cliente, como fallback para dados cadastrais — nunca para regime tributário.** Em `integrations.ts` (`consultarCNPJReceitaWS`, endpoint real `https://receitaws.com.br/`) a chamada serve para resolver endereço / Inscrição Estadual de um CNPJ, acionada **como fallback após a BrasilAPI**. Ela **não** consulta Simples Nacional e não participa da emissão de pedido. *Por quê:* A integração existe para completar o cadastro quando a fonte primária (BrasilAPI) falha; regime tributário não faz parte desse escopo hoje. *Regressão:* Reaproveitar essa chamada para tentar inferir regime Simples, ou torná-la primária (antes da BrasilAPI), quebrando a ordem de fallback e o contrato de custo/latência da criação de cliente.
3. **A seleção de natureza de operação no envio ao Tiny usa exclusivamente `tiny_valor`.** Em `tiny-enviar-pedido-venda-v1`, a natureza é resolvida a partir dos dados já persistidos no pedido (`natureza_id` / nome de `natureza_operacao`), buscando **sempre** o mapeamento em `tiny_empresa_natureza_operacao` e lendo o campo `tiny_valor`. A coluna `tiny_valor_simples` existe no schema mas **nunca é lida**. Não há detecção de dual-mapping, nem short-circuit, nem ramo condicional por optante. *Por quê:* Hoje o envio confia inteiramente no que já foi decidido/persistido no pedido; a escolha de natureza é única e determinística por mapeamento cadastrado. *Regressão:* Introduzir leitura de `tiny_valor_simples` ou ramos por optante **sem** também implementar a consulta/validação de regime — isso enviaria natureza divergente sem a fonte de verdade que a justifica.
4. **CNPJ deve ser mascarado em logs — hoje isso é apenas parcial e precisa ser corrigido.** `tinyERPSync.ts` e `tiny-enviar-pedido-venda-v1` não logam CNPJ diretamente; porém `integrations.ts` e `CNPJTestTool.tsx` fazem `console.log` de `cpf_cnpj` em texto claro. A regra de negócio desejada é mascarar (preservar apenas os primeiros e últimos dígitos), mas o código atual **viola isso** nesses dois pontos. *Por quê:* CNPJ é identificador sensível de empresa; logs são consultados por múltiplos usuários e podem vaziar. Mascarar reduz risco sem perder rastreabilidade. *Regressão:* Adicionar novos pontos de log com `cpf_cnpj` em texto claro, ou remover o mascaramento onde ele vier a ser aplicado. (Débito atual: corrigir `integrations.ts` e `CNPJTestTool.tsx`.)

Regras propostas e REFUTADAS pela verificação (não viram regra de negócio ativa): - *Consulta obrigatória ao Simples no envio, com bloqueio se falhar (RN-001):* **FALSA** — não há revalidação ReceitaWS no envio; `tiny-enviar-pedido-venda-v1` não consulta regime nem bloqueia. - *Cache de `optante_simples_nacional` com timestamp, revalidado antes do ERP (RN-003):* **IMPRECISA** — a coluna existe no `schema_baseline.sql` mas nunca é populada, consultada ou revalidada por nenhuma Edge Function/migration. - *Hierarquia `tiny_valor_simples` vs `tiny_valor` por optante (RN-004):* **FALSA** — não existe essa seleção; `tiny_valor_simples` nunca é lido. - *Não persistir resposta bruta da ReceitaWS de regime (RN-005):* **VAZIA** — ReceitaWS de regime nunca é chamada,

então não há resposta bruta a persistir. - *Dual-mapping detectado por `tiny_valor_simples` preenchido (RN-006): **FALSA*** — não há detecção de dual-mapping em lugar nenhum. - *Timeout de 5s na ReceitaWS (RN-007): **NÃO IMPLEMENTADO*** — `consultarCNPJReceitaWS` não define timeout (fetch padrão ~30s), e não há chamada de regime. - *Short-circuit sem dual-mapping pula ReceitaWS (RN-009): **FALSA*** — toda requisição busca sempre o mapeamento Tiny, sem short-circuit. - *Retry único em HTTP 429 após 3s (RN-010): **NÃO IMPLEMENTADO*** — não há lógica de retry de regime. - *Bloqueio D3 + `REGIME_LOOKUP_FAILED` no envio (RN-011): **FALSA*** — nunca há consulta de regime na emissão, logo o erro nunca é retornado; frontend (`SalesPage.tsx`) não o trata.

Dúvidas em aberto

As dúvidas abaixo não têm resposta conclusiva no código atual e envolvem a divergência entre histórico Git / contrato e o working tree. As demais dúvidas do backlog (DV-001 a DV-011) já foram respondidas pela verificação e estão registradas como [RESPONDIDA].

1. **Dessincronização histórico ↔ working tree.** Os commits `7bc21f1` (V1.68) e `c0bef7c` (V1.67), de ~2026-06-25, descrevem revalidação de Simples, bloqueio D3, tabela `regime_lookup_falha` e detecção de dual-mapping — mas **nada disso existe no código atual**. As features estão em branch experimental? Foram revertidas? O history está dessincronizado do working tree? *Como resolver:* código (comparar branches / `git log --all` das funções `tiny-enviar-*`, `integrations.ts` e migrations 138+) e **cliente** (confirmar se a feature foi descartada ou está planejada).
2. **Coluna órfã `optante_simples_nacional`.** Existe em `schema_baseline.sql:88-89` (snapshot de produção) mas **sem migration criadora** entre 001-143, e não é populada nem lida por nenhum código. Como ela entrou em produção? O `schema_baseline` está desatualizado, ou migrations foram removidas? *Como resolver:* código (auditar migrations vs. schema baseline) e **cliente** (confirmar estado real do banco de produção).
3. **`tiny_valor_simples` nunca usado.** A coluna existe em `tiny_empresa_natureza_operacao` (`schema_baseline.sql:682`) mas o seletor de natureza no envio a ignora por completo. É código não-finalizado, preparação para feature futura, ou resíduo? *Como resolver:* cliente (confirmar intenção de produto para dual-mapping fiscal).
4. **Distinção entre `NULL` “nunca consultado” vs. “consulta inconclusiva” (DV-012).** Caso a feature de regime seja ativada, `optante_simples_nacional = NULL` pode significar três coisas (nunca consultado / cliente PF / consulta falhou). Não há coluna/flag de motivo para desambiguar. Deve haver? Como o negócio quer diferenciar esses casos? *Como resolver:* código (verificar se há coluna de timestamp/flag prevista) e **cliente** (definir se a desambiguação importa para a regra de natureza).
5. **Onde a revalidação de regime deveria ocorrer, se implementada.** O contrato diz “antes do envio ao ERP” (best-effort na criação, bloqueante no envio), mas nenhuma função faz isso hoje. Em qual fase o negócio realmente quer a validação? *Como resolver:* cliente (decisão de produto sobre o momento da validação).

Dúvidas do backlog já respondidas pela verificação

- **DV-001 [RESPONDIDA]:** `optante_simples_nacional` **existe** na tabela `cliente` (`schema_baseline.sql:88`), porém é campo órfão — sem migration criadora e nunca populado/consultado.
- **DV-002 [RESPONDIDA]:** `consultarSimplesNacional` / `receitaws-client.ts` **não existem**. Há `consultarCNPJReceitaWS` em `integrations.ts`, mas para CNPJ/IE, não para Simples.
- **DV-003 [RESPONDIDA]:** `tipoPessoa` vem de `tipo_pessoa_nome` (RPC) ou fallback pelo comprimento do documento (11→PF, 14→PJ). Nunca por ReceitaWS.
- **DV-004 [RESPONDIDA]:** **Não há** COUNT/head para detectar dual-mapping no envio; `tiny-enviar-pedido-venda-v1` busca sempre o mapeamento, sem short-circuit.

- **DV-005 [RESPONDIDA]:** Sem consulta de regime no envio, não há timeout nem bloqueio D3 por timeout de regime.
- **DV-006 [RESPONDIDA]:** `create-cliente-v2` (migration 114) **não** chama ReceitaWS para regime nem tem parâmetro `p_optante_simples_nacional`.
- **DV-007 [RESPONDIDA]:** Tabela `regime_lookup_falha` **não existe** (migration 138 do commit `c0bef7c` não está no repo). Falhas de regime não são auditadas.
- **DV-008 [RESPONDIDA]:** Frontend **não** `exibe` `optante_simples_nacional`; `mapClienteCompleto` não retorna o campo. (Estados “Não consultado”/“Indisponível” citados em `7bc21f1` não estão no arquivo atual.) — vale confirmação visual por Playwright se/quando a feature existir.
- **DV-009 [RESPONDIDA]:** `REGIME_LOOKUP_FAILED` nunca é retornado (regime nunca consultado na emissão); `SalesPage.tsx` não trata esse erro nem oferece botão de retry.
- **DV-011 [RESPONDIDA]:** A integração ReceitaWS é **real** (`https://receitaws.com.br/` em `integrations.ts:214`), não mockada — mas usada só para lookup de CNPJ, não de regime Simples.

Condições de Pagamento

Fonte de verdade: `supabase/functions/condicoes-pagamento-v2/index.ts`, `src/types/condicaoPagamento.ts`, tabela `Condicao_De_Pagamento`, e as funções de emissão (`emitirpedido`, `emitir-pedido-sem-vendedor`, `tiny-enviar-pedido-v1`). As regras abaixo foram conferidas contra o código; onde o spec original divergia da implementação, a regra foi corrigida ou marcada como comportamento atual (não como intenção validada).

Regras de negócio

R01 — Intervalo de parcelas define os vencimentos

Cada condição de pagamento tem um campo `intervalo_parcela` (array de dias) que define quantos dias após a data do pedido cada parcela vence. Ele é lido diretamente na emissão do pedido ao Tiny. *Por quê:* a emissão precisa saber, por parcela, o número de dias de vencimento para gerar as parcelas e o fluxo de caixa correto. *Regressão:* se `intervalo_parcela` vier vazio/ `NULL` na emissão, o `.map()` / `.length` sobre ele gera zero parcelas ou datas erradas. *Correção vs spec:* **NÃO existe validação** que garanta `intervalo_parcela` não-vazio quando `Quantidade_parcelas > 1`. O CREATE só checa se `prazoPagamento` está vazio (linha 291), não se contém números válidos. Entrada como `'abc/def'` passa na checagem e `processarPrazoPagamento` devolve `intervaloParcela: []` silenciosamente, criando registro inconsistente sem erro. Ver D11 nas dúvidas.

R02 — Quantidade de parcelas é derivada do array

`Quantidade_parcelas` é sempre `intervalo_parcela.length` — na API v2 os dois são calculados juntos por `processarPrazoPagamento` a partir do mesmo `prazoPagamento`. *Por quê:* usado para dividir o valor do pedido entre parcelas e para exibição/filtros. *Regressão:* se `Quantidade_parcelas` divergir de `intervalo_parcela.length`, a divisão de valor por parcela e os loops de emissão usam contagem incorreta. *Ressalva:* essa igualdade só é imposta pela **API v2**, não pelo banco (ver R14).

R03 — `intervalo_parcela` é array de números, não string

`intervalo_parcela` é persistido como array (ex: `[10, 15, 20]`) e consumido diretamente na emissão.

Por quê: dados estruturados e reutilizáveis, sem reparse em tempo de emissão. *Regressão:* se fosse gravado como string `'10/15/20'`, o código de emissão quebraria ao chamar `.map()` / `.length`. *Observação:* o parser aceita `parseFloat` (linha 99), então valores não-inteiros como `10.5` seriam aceitos apesar do domínio ser “dias”.

R04 — Desconto é percentual aplicado uma vez sobre o total

`Desconto` é um percentual aplicado uma única vez ao valor total: `desconto = valorTotal * (Desconto/100)`. *Por quê:* cálculo previsível e transparente no faturamento. *Regressão:* se interpretado como valor fixo em R\$, um “10” viraria R\$10 em vez de 10%, quebrando pedidos acima de R\$100. *Ressalva:* não há bounds check — `Desconto = 150` ou `-10` são aceitos e usados no cálculo sem validação de range (ver D13).

R05 — `Parcelamento = (Quantidade_parcelas > 1)`

`Parcelamento` é `TRUE` quando `Quantidade_parcelas > 1`, `FALSE` caso contrário (à vista / 1 parcela).

Por quê: flag booleano para interface e filtros de condições parceladas. *Regressão:* se ficar `FALSE` para uma condição de 3 parcelas, filtros que usam o flag ignoram a condição incorretamente. *Ressalva:* recalculado no UPDATE apenas quando `prazoPagamento` é enviado (linha 381). Um UPDATE que muda só desconto não recalcula `Parcelamento` — mas como o intervalo também não muda nesse caso, o flag permanece coerente. Inconsistência real só ocorre via escrita SQL direta ou parsing quebrado (ver D02).

R06 — Descrição automática usa apenas o ÚLTIMO prazo (corrige spec)

Quando a descrição não é fornecida, `gerarDescricao()` a monta como `${forma} - ${prazo} dias - desc extra ${desconto}%`, usando apenas o último valor do intervalo (`prazoPagamento`), não todas as parcelas. *Por quê:* nomeação consistente e busca na interface. *Correção vs spec:* o spec afirmava que a descrição inclui todas as parcelas (`10/15/20`). Isso é falso. Para entrada `10/15/20`, a descrição gerada é `"PIX - 20 dias - desc extra 0%"` — as parcelas intermediárias (10, 15) ficam ocultas. Se a descrição legível de todas as parcelas for requisito de auditoria, é um gap: só `formatarPrazoPagamento()` (em `condicaoPagamento.ts`) produz o formato `3x (10 / 15 / 20 dias)`, e ele não é usado na geração da descrição salva.

R07 — Forma de pagamento obrigatória; meio de pagamento opcional

`forma_pagamento_id` é obrigatório no CREATE (linhas 287–289); `meio_pagamento` é opcional e mais granular (instituição). *Por quê:* forma define a categoria ampla (PIX vs transferência); meio detalha a instituição. *Regressão:* sem `forma_pagamento_id`, a emissão não determina o método de pagamento. *Ressalva:* no UPDATE não há como limpar `forma_pagamento_id` (omitir mantém o valor; não existe caminho para setar `NULL`). Assimetria CREATE/UPDATE (ver D14).

R08 — Valor mínimo é apenas metadado (não enforcado na emissão)

`valor_minimo` é o valor total mínimo do pedido para a condição ser válida. *Por quê:* condições de prazo longo (30/60/90) podem exigir um piso de pedido. *Correção vs spec:* nenhuma função de emissão valida `valor_minimo`. `emitirpedido` e `emitir-pedido-sem-vendedor` só selecionam `intervalo_parcela`, `Desconto` da condição — não leem `valor_minimo`. Não há `if (valorPedido < condicao.valor_minimo) throw`. Hoje o campo é informativo (eventual enforcement só existiria em UI/RLS/trigger — não confirmado). Ver D05.

R09 — Condição de crédito é apenas metadado (não enforcada na emissão)

`Condição_de_crédito` (boolean) marca condições que, por regra de negócio, exigiriam aprovação de crédito. *Por quê:* vendas parceladas podem demandar análise adicional. *Correção vs spec:* **nenhuma função de emissão bloqueia** quando `Condição_de_crédito = true`. O campo é salvo e retornado pela API, mas não dispara validação de score no código de emissão. Hoje é metadado. Ver D07.

R10 — Prazo é processado em array no CREATE, mas conteúdo não é validado

No CREATE/UPDATE, `prazoPagamento` (string) é convertido por `processarPrazoPagamento`: `'30/60/90' → [30, 60, 90]`; `'' → []` (à vista, 1 parcela, prazo 0). *Por quê:* estrutura os dados de entrada em array processável na escrita. *Correção vs spec:* a validação de **conteúdo numérico não é aplicada** na API v2. O validador com regex (`validarPrazoPagamento` em `condicaoPagamento.ts`, que rejeita `abc` e exige ordem crescente) **existe mas não é chamado** pela edge function. Só a checagem de string vazia (linha 291) roda. Logo `'abc/def'` passa e vira `intervaloParcela: []`. Ver D11.

R11 — Descrição regenerada no UPDATE mistura valores novos (body) + antigos (banco pré-UPDATE)

Se o UPDATE altera `formaPagamentoId`, `prazoPagamento` ou `descontoExtra` **sem** fornecer `descricao`, a descrição é regenerada. Para cada componente **não** enviado no body, o valor é buscado no banco **antes** de aplicar o UPDATE. *Por quê:* mantém a descrição sincronizada sem exigir reenvio de todos os campos. *Regressão / comportamento contra-intuitivo:* `gerarDescricao` recebe uma mistura — ex: novo prazo (do body) + desconto antigo (buscado do banco pré-UPDATE, linhas 416–426). O resultado reflete o estado pós-update dos campos enviados, mas o estado pré-update dos campos omitidos. Quando prazo e desconto mudam juntos mas só um vem no body, a descrição pode não bater com o estado final da linha.

R12 — `Prazo_pagamento` (escalar) = último valor do intervalo

`Prazo_pagamento` é sempre o último elemento de `intervalo_parcela` (ex: `10/15/20 → 20`). *Por quê:* simplifica consultas que só precisam do prazo máximo (ex: política de crédito por dias máximos). *Regressão:* se não acompanhar o último intervalo, análises de prazo ficam incorretas. Mantido em sincronia por `processarPrazoPagamento` no CREATE/UPDATE.

R13 — Divisão de parcelas com ajuste na última (fechamento do total)

O valor total é dividido igualmente entre os intervalos, com a última parcela ajustada para fechar o total (arredondamento). *Por quê:* garante soma das parcelas = valor total, sem centavos perdidos. *Regressão:* sem ajuste na última, a soma poderia divergir do total em centavos. *Nota:* a lógica de emissão vive nas funções `emitir*`, não na `condicoes-pagamento-v2`; confirmar o cálculo exato ao mexer nessas funções.

R14 — Sincronização `Quantidade_parcelas === intervalo_parcela.length` só na API v2

A API v2 sempre calcula ambos os campos juntos a partir de `prazoPagamento`, garantindo a igualdade em CREATE e em UPDATE que envia `prazoPagamento`. *Por quê:* evita divergência entre os dois campos que descrevem a mesma coisa. *Correção vs spec / ressalva:* **não há CHECK CONSTRAINT no banco** e a RPC alternativa (`rpc_insert_condicao_pagamento`) aceita `quantidade_parcelas` e `intervalo_parcela` como parâmetros **independentes**, permitindo violação (ex: `quantidade_parcelas=3` com `intervalo=[10,20]`). Escritas SQL diretas também escapam da sincronização. A garantia é só a nível de código da v2.

Dúvidas em aberto

As dúvidas cujo comportamento já está claro no código foram absorvidas nas regras acima e marcadas [RESPONDIDA]. Permanecem abertas as que dependem de decisão de negócio ou verificação em ambiente.

D04 — Semântica de “à vista”: [0] vs [] — resolver com o cliente

`prazoPagamento='0'` produz `{quantidadeParcelas:1, prazoPagamento:0, intervaloParcela:[0]}`, enquanto `''` produz `intervaloParcela:[]`. Ambos representam “à vista”, mas geram arrays diferentes. **Como resolver:** decisão de negócio (cliente). Definir qual é a forma canônica de “à vista” e, se necessário, normalizar no `processarPrazoPagamento`.

D05 — `valor_minimo` deveria ser enforcado na emissão? — verificar em ambiente

[Parcialmente RESPONDIDA no código] Confirmado que **nenhuma função de emissão valida** `valor_minimo` (ver R08). Aberto: é intencional (campo puramente informativo) ou falta enforcement? **Como resolver: Playwright** — tentar emitir um pedido abaixo do `valor_minimo` de uma condição e observar se algo (UI/RLS/trigger) bloqueia. Se nada bloquear, confirmar com o cliente se deveria bloquear.

D07 — `Condição_de_crédito` deveria bloquear emissão sem aprovação? — verificar em ambiente

[Parcialmente RESPONDIDA no código] Confirmado que **nenhuma emissão bloqueia** por esse flag (ver R09). Aberto: é metadado por design ou deveria disparar validação de score/aprovação? **Como resolver: Playwright** — emitir pedido usando condição com `Condição_de_crédito=true` sem aprovação prévia e observar se é bloqueado. Confirmar intenção com o cliente.

D11 — CREATE deveria usar `validarPrazoPagamento` (regex + ordem crescente)? — decisão de negócio/código

O validador robusto existe em `condicaoPagamento.ts` mas **não é chamado** pela edge function; só a checagem de string vazia roda, permitindo `'abc/def'` → `intervalo_parcela: []` sem erro. **Como resolver: código** — decidir se a v2 deve chamar `validarPrazoPagamento` antes de `processarPrazoPagamento` (rejeitando formato inválido e ordem não-crescente). Envolve decisão de negócio sobre rejeitar vs normalizar.

D12 — Registro “quebrado por parsing” x “à vista real” — decisão de negócio

Entrada inválida (`'abc'`) resulta em `quantidadeParcelas=1`, `Parcelamento=FALSE`, `intervalo_parcela=[]` — indistinguível de uma condição à vista legítima, embora seja falha de parsing. **Como resolver:** ligada a D11. Se D11 for resolvida com validação de conteúdo, D12 desaparece. Requer decisão do cliente sobre o tratamento.

D13 — `Desconto` sem validação de range (0–100) — decisão de negócio/código

`Desconto=150` ou `-10` são aceitos e entram no cálculo `valorTotal * (Desconto/100)` sem bounds check. **Como resolver: código** — adicionar validação de faixa `0 <= Desconto <= 100`, se o negócio confirmar que esse é o intervalo válido.

D14 — Não é possível limpar `forma_pagamento_id` /campos via UPDATE — decisão de design

UPDATE só altera campos presentes no body; não há caminho para setar `forma_pagamento_id` como `NULL`. Como `forma_pagamento_id` é obrigatório (R07), na prática isso não é um problema para esse campo, mas a assimetria existe para campos opcionais (ex: `meio_pagamento`). **Como resolver: código/design** — decidir se UPDATE deve suportar limpar campos opcionais explicitamente (ex: enviar `null`).

D15 — Divergência entre rotas de escrita (API v2 x RPC x SQL direto) — decisão de arquitetura

A API v2 sincroniza `Quantidade_parcelas` e `intervalo_parcela`; a RPC `rpc_insert_condicao_pagamento` e SQL direto não. Sem CHECK CONSTRAINT, o banco não protege a invariante. **Como resolver: código/banco** — avaliar adicionar CHECK CONSTRAINT (`array_length(intervalo_parcela,1) = Quantidade_parcelas`) ou eliminar/ajustar a RPC para forçar a mesma sincronização. Decisão de arquitetura sobre qual rota é canônica.

Comissão

Regras de negócio

- Pedidos bonificados não geram comissão.** Quando `pedido_venda.natureza_operacao = 'Bonificação'`, `generate_vendedor_comissao` interrompe e retorna status `'bonificacao_sem_comissao'`.
 - Por quê:* Bonificações são operações de cortesia ou promoção, não representam venda real, então não devem comissionar o vendedor.
 - Regressão:* Um pedido com `natureza_operacao = 'Bonificação'` gera comissão em `vendedor_comissao`.
 - Nota (verificação):* A comparação é **case-sensitive e sensível a acento** (migration 143, linha 57: `if v_pedido.natureza_operacao = 'Bonificação'`, sem `LOWER()` nem transliteração). Variações como `'bonificação'` ou `'BONIFICACAO'` contornam a regra. Ver Dúvida 6.
- Pedidos excluídos (soft-delete) não geram comissão.** Quando `pedido_venda.deleted_at IS NOT NULL`, `generate_vendedor_comissao` interrompe e retorna `'pedido_deletado_sem_comissao'` (migration 143, linhas 43-46).
 - Por quê:* Pedido deletado deixou de ser válido para comissionar; evita pagar comissão sobre transações canceladas.
 - Regressão:* Um pedido com `deleted_at != NULL` gera comissão em `vendedor_comissao`.
 - Nota (verificação):* Isto impede **criar** comissão nova em pedido já deletado, mas **não reverte** comissões já existentes (ver Regra 11).
- Toda comissão deve ter um vendedor associado.** Uma comissão sem `vendedor_uuid` (ou com `pedido_venda.vendedor_uuid` NULL quando a comissão deveria ser gerada) é órfã.
 - Por quê:* Comissão sem vendedor não pode ser paga nem rastreada; invalida o fluxo de pagamento.
 - Regressão:* Uma comissão é criada sem `vendedor_uuid`.
- Comissão é calculada por faixa de desconto do cliente (Tipo 1) ou alíquota fixa (Tipo 2).** O campo `Comissão` do vendedor define o modelo; valores diferentes de 1 e 2 lançam exceção `'Tipo de comissão inválido'`.
 - Por quê:* Dois modelos coexistem: clientes premium (desconto variável) pagam comissão por tabela; outros vendedores recebem percentual fixo.
 - Regressão:* Um vendedor com `Comissão != 1` e `!= 2` gera comissão sem que o sistema lance exceção.
- Tipo 1 (por lista): a comissão é buscada em `listas_preco_comissionamento` pela faixa onde `desconto_minimo <= desconto_cliente <= desconto_maximo`.** Quando nenhuma faixa encaixa, o percentual é definido como 0 (migration 143, linhas 94-96: `if v_percentual is null then v_percentual := 0; end if;`).

- *Por quê*: Permite estruturar incentivos alinhando o interesse do vendedor com a margem.
 - *Regressão*: Desconto do cliente encaixa em uma faixa mas a comissão sai como 0, ou encaixa em faixa errada.
 - *Nota (verificação)*: Corrigida a redação original. Percentual = 0 quando não há faixa **não é falha silenciosa**, é o comportamento esperado. Só há degradação problemática quando o motivo do “nenhum match” é dado ausente (lista NULL) — ver Regra 6 e Dúvida 5.
- 6. **Tipo 1: cliente é obrigatório; lista de preço ausente resulta em percentual 0.** Se o cliente não existir, `generate_vendedor_comissao` lança exceção (migration 143, linhas 76-84). Se `lista_preco_id` do cliente for NULL ou não houver faixas, o percentual resolve para 0 (degradação graceful, sem bloquear o pedido).
 - *Por quê*: Cliente é FK obrigatória; lista ausente não deve bloquear o faturamento.
 - *Regressão*: Cliente inexistente não gera exceção; ou lista ausente bloqueia o pedido em vez de comissionar 0.
- 7. **Tipo 2: comissão = `aliquotafixa` do vendedor; se NULL, defaulta para 0.**
 - *Por quê*: Modelo simples para vendedores com taxa fixa independente do cliente; NULL é tratado como sem direito a comissão.
 - *Regressão*: Vendedor Tipo 2 com `aliquotafixa = NULL` tem comissão calculada como percentual não-zero.
- 8. **Valor da comissão = `ROUND(valor_total * percentual / 100, 2)`.** (migration 143, linha 104: `v_valor_comissao := round((v_pedido.valor_total::numeric * v_percentual / 100), 2)`).
 - *Por quê*: Precisão monetária em duas casas (centavos) para pagamento e auditoria.
 - *Regressão*: O valor é arredondado com 3+ casas ou truncado, acumulando divergências centavo a centavo.
- 9. **Quando o pedido é faturado (`status = 'Faturado'`), a comissão é gerada automaticamente.**
 - *Por quê*: Faturamento é o marco de realização da venda, momento em que nasce o direito à comissão.
 - *Regressão*: Pedido muda para `'Faturado'` mas a comissão não aparece em `vendedor_comissao`.
- 10. **Quando `valor_total` do pedido é alterado para valor > 0, a comissão é recalculada.**
 - *Por quê*: Mudanças no valor da venda devem refletir na comissão em tempo real para manter a auditoria correta.
 - *Regressão*: `valor_total` muda de 100 para 200 mas `valor_comissao` continua calculado sobre 100.
- 11. **Não há mecanismo automático de reversão/cancelamento de comissão quando `valor_total` vai para 0 ou o pedido é deletado.** A migration 143 impede **criar** comissão nova em pedido já deletado, mas não existe trigger, `ON DELETE CASCADE` ou RPC que **delete** ou **reverta** comissões já criadas.
 - *Por quê*: Desvio de design conhecido: o sistema registra comissão no lançamento mas não a reversa se o pedido cai.
 - *Regressão*: Pedido é deletado mas a comissão permanece em `vendedor_comissao` com o valor original. (Comportamento **confirmado** pela verificação — este é o estado atual, não um bug corrigível apenas por documentação.)
- 12. **Quando múltiplas faixas de desconto encaixam, a primeira por `ORDER BY desconto_minimo ASC, id ASC` é usada** (migration 143, linha 91).
 - *Por quê*: Desambiguação determinística: mesmo cenário sempre produz o mesmo cálculo.
 - *Regressão*: O `ORDER BY` é alterado para DESC, ou `id` é removido do critério, mudando comissões entre clientes.
- 13. **`desconto_minimo <= desconto_maximo` em `listas_preco_comissionamento` (constraint CHECK).**

- *Por quê*: Valida integridade da faixa; evita faixas invertidas com comportamento imprevisível.
 - *Regressão*: A constraint é removida e faixas com `min > max` são inseridas, causando comissões erráticas.
14. **Período tem formato YYYY-MM (ex.: '2025-10') armazenado como TEXT.**
- *Por quê*: Padrão consistente para filtros e agrupamentos.
 - *Regressão*: Formato muda para YYYYMM ou YYYY/MM e queries que filtram por `periodo` quebram silenciosamente.
 - *Nota (verificação)*: Só o formato **mensal** é implementado. O suporte a período anual ('2025') aparece nos tipos TypeScript mas **não existe** no banco nem nas queries — ver Dúvida 14.
15. **Período de um vendedor é único por mês (UNIQUE controle_comissao_periodo(vendedor_uuid, periodo))** (migration 083, linha 114).
- *Por quê*: Cada vendedor tem no máximo um registro de controle por período.
 - *Regressão*: A constraint é removida e múltiplos registros por período causam ambiguidade em saldo e status.
 - *Nota (verificação)*: Esta constraint **não** impede múltiplos períodos com status 'aberto' (períodos diferentes) — ver Regra 16 abaixo / Dúvida 3 [RESPONDIDA].
16. **Status de período é um de: 'aberto', 'fechado', 'pago' (constraint CHECK).** Não há limite para quantos períodos 'aberto' um vendedor pode ter ao mesmo tempo.
- *Por quê*: Estados bem definidos controlam transições e impedem estados inválidos que quebrariam relatórios.
 - *Regressão*: Status muda para valor inválido (ex.: 'aguardando') e a constraint falha; ou, se removida, inconsistências propagam.
 - *Nota (verificação)*: O status 'pago' é **lido** em relatórios mas **nunca escrito** por nenhuma função — ver Dúvida 1/12.
17. **Não é possível adicionar lançamentos manuais via RPC em período com status 'fechado' ou 'pago'.** `create_lancamento_comissao_v2` valida o status (migration 084, linhas 118-124).
- *Por quê*: Período fechado é imutável para auditoria; pagamento liquidado não admite lançamento retroativo.
 - *Regressão*: A RPC passa a permitir inserção em período fechado (validação removida ou `status = NULL`).
 - *Nota (verificação)*: Esta proteção é **contornável** pelos endpoints PUT/DELETE de `comissoes-v2` , que operam direto na tabela sem passar pela RPC — ver Dúvida 4 [RESPONDIDA] e Regra 21.
18. **Saldo final = saldo_anterior + total_comissao + total_creditos - total_debitos - total_pagos .**
- *Por quê*: Fórmula contábil: saldo anterior carregado, somado de créditos da venda, deduzido de débitos e pagamentos realizados.
 - *Regressão*: O operador de débito muda para `+` e saldos crescem em vez de diminuir com devoluções.
19. **Novo período herda saldo_anterior = saldo_final do período anterior.**
- *Por quê*: Carryover de saldo; o acúmulo entre períodos reflete débito/crédito contínuo do vendedor.
 - *Regressão*: A herança não acontece, `saldo_anterior` do novo período fica 0 e o histórico acumulado é perdido.
20. **Preview de comissões exclui pedidos que já têm comissão gerada (NOT EXISTS vendedor_comissao).**
- *Por quê*: Evita contar comissões duas vezes; o preview mostra oportunidades futuras, não passadas.

- *Regressão*: O check `NOT EXISTS` é removido e o preview lista comissões já geradas, confundindo o usuário com duplicatas.
21. **Backoffice é o único que pode criar, editar, deletar lançamentos e pagamentos e fechar períodos.**
- *Por quê*: Segregação de dever: vendedor vê sua comissão, backoffice gerencia; previne manipulação pelo vendedor.
 - *Regressão*: O role check é removido em endpoints POST/PUT/DELETE e o vendedor cria lançamentos próprios fictícios.
22. **Vendedor vê apenas suas próprias comissões via RLS; backoffice vê todas.**
- *Por quê*: Isolamento de dados: um vendedor não acessa comissão de colega; backoffice tem visão completa.
 - *Regressão*: A policy RLS é desabilitada ou o filtro removido e o vendedor lê comissões de colegas.
 - *Nota (verificação)*: A migration 143 (INSERT em `vendedor_comissao`) **não especifica** a policy RLS de leitura por vendedor; o escopo exato precisa ser confirmado — ver “Nova dúvida 6”.
23. **`oc_cliente` (ordem de compra) e `cliente_nome` são gravados (desnormalizados) em cada comissão** (migration 143, linhas 139-140).
- *Por quê*: Auditoria: o relatório mostra OC e cliente mesmo se forem deletados depois.
 - *Regressão*: Os campos não são populados e o relatório fica em branco para OC e Cliente (como no bug de abr/2026).
24. **Comissão é gerada como INSERT (nova) ou UPDATE (pedido já tem comissão)** (migration 143, linhas 113-123).
- *Por quê*: Idempotência: múltiplas chamadas de `generate_vendedor_comissao` não criam duplicatas, atualizam a existente.
 - *Regressão*: A lógica de idempotência é removida e a mesma comissão é inserida várias vezes.

Regra removida

- **Faixa de desconto com (`min=0, max=0, comissao=0`) é automaticamente deletada após INSERT — REFUTADA pela verificação.** Não existe trigger de autolimpeza. O código apenas **lê e filtra** essas faixas no cálculo; nenhum mecanismo as remove da tabela. A regressão descrita (faixa de zeros absorve `desconto=0` como primeiro match e zera comissões) é um risco **real** justamente porque a limpeza automática não existe — depende de as faixas de zeros não serem inseridas / serem removidas manualmente.

Dúvidas em aberto

As dúvidas abaixo que já têm resposta no código estão marcadas **[RESPONDIDA]**. As sem resposta clara precisam de decisão do cliente ou verificação em ambiente.

1. **Transição `'fechado'` → `'pago'` : é automática (quando `total_pagos >= saldo_final`) ou manual?** — *Resolver via cliente.* [RESPONDIDA parcialmente pelo código]
`fechar_periodo_comissao_v2` só grava `'fechado'` (migration 084, comentário “Por enquanto apenas registra o pagamento”). O status `'pago'` é **lido** em relatório mas **nunca escrito**. Processo incompleto: falta definir com o cliente o critério e implementar a transição.
2. **Comissão de pedido soft-deletado permanece órfã ou é removida?** — *Resolver via código.* [RESPONDIDA] Permanece órfã. A migration 143 impede criar comissão em pedido deletado, mas não há trigger/ `ON DELETE CASCADE` que remova comissões já existentes. Ver Regra 11.

3. **Um vendedor pode ter múltiplos períodos 'aberto' simultaneamente?** — *Resolver via Playwright/código.* [RESPONDIDA] Sim. `UNIQUE(vendedor_uuid, periodo)` só impede dois registros do mesmo período; `2026-01` e `2026-02` podem estar 'aberto' ao mesmo tempo. Ver Regra 16.
4. **PUT/DELETE de lançamentos e pagamentos em `comissoes-v2` validam se o período está 'fechado' / 'pago'?** — *Resolver via código.* [RESPONDIDA] Não — **vulnerabilidade confirmada.** Os endpoints PUT (linhas ~204-237) e DELETE de `comissoes-v2/index.ts` chamam `.update()` / `.delete()` direto na tabela, sem passar pela RPC `create_lancamento_comissao_v2`, contornando a validação de período. Requer decisão: replicar a validação nos endpoints.
5. **Se `cliente.lista_de_preco` ficar NULL após a comissão já ter sido gerada (Tipo 1), a comissão recalcula para 0 na próxima alteração do pedido?** — *Resolver via código.* [RESPONDIDA] Sim, silenciosamente. `generate_vendedor_comissao` faz UPDATE quando a comissão já existe; com lista NULL nenhuma faixa encaixa, percentual = 0, e a comissão preexistente é **sobrescrita para 0** sem erro. Comportamento a validar com o cliente se é desejável.
6. **A verificação de `natureza_operacao = 'Bonificação'` é case-sensitive? Variações contornam a regra?** — *Resolver via cliente.* [RESPONDIDA no código, decisão pendente com cliente] É comparação exata (sem `LOWER()` /transliteração). `'bonificação'` ou `'BONIFICACAO'` **geram comissão indevidamente.** Definir com o cliente se o valor é controlado (dropdown fixo) ou livre — se livre, normalizar a comparação.
7. **A omissão de `ORDER BY id` na API `listas-preco-v2` (linha 211) afeta o cálculo de comissão?** — *Resolver via código.* [RESPONDIDA — não é vulnerabilidade] O cálculo usa a RPC `generate_vendedor_comissao`, que já ordena por `desconto_minimo ASC, id ASC`. A API só lista faixas para exibição/preview; o impacto é apenas de ordenação na UI, não no valor calculado.
8. **Se backoffice edita `saldo_anterior` / `saldo_final` de um período 'fechado', os relatórios recalculam com o novo valor?** — *Resolver via código.* [RESPONDIDA] Sim — **vulnerabilidade confirmada.** A RLS de backoffice é `FOR ALL` sobre `controle_comissao_periodo` e não há trigger de imutabilidade; UPDATE em período 'fechado' é permitido e os relatórios usam o novo valor. Requer decisão: bloquear edição de períodos fechados.
9. **Qual o propósito do campo `debito` (boolean) em `vendedor_comissao`?** — *Resolver via código/cliente.* [RESPONDIDA — legado] Sempre inserido como `false` (migration 143), nunca populado dinamicamente. O crédito/débito real fica em `lancamentos_comissao.tipo`. Aparece ser coluna legada sem uso; confirmar antes de remover.
10. **Pedido com comissão, depois soft-deletado e restaurado (`deleted_at` → NULL): a comissão é recriada?** — *Resolver via cliente.* [RESPONDIDA no código, decisão pendente] Não automaticamente. Nenhum trigger chama `generate_vendedor_comissao` na restauração; o pedido pode ficar 'Faturado' sem comissão. Só uma chamada manual (webhook/RPC) recria. Definir com o cliente o comportamento esperado.
11. **O PUT `/vendas` permite editar `periodo` de uma `vendedor_comissao`, transferindo comissão entre períodos sem revalidar saldos?** — *Resolver via cliente.* [RESPONDIDA no código, decisão pendente] Sim (linhas ~417-421 de `comissoes-v2/index.ts`). A troca de período não recalcula saldos do período de origem nem do destino — discrepância contábil silenciosa. Definir se a edição deve ser bloqueada ou disparar recálculo em cascata.
12. **Qual o passo a passo esperado do backoffice para declarar um período totalmente pago? Existe botão "Marcar como Pago"?** — *Resolver via cliente.* [RESPONDIDA parcialmente] A transição para 'pago' não é invocada em lugar nenhum. Provavelmente o status é documentado mas não usado na prática. Confirmar com o cliente o fluxo desejado (relacionado à Dúvida 1).

13. **Lançamento com `descricao` string vazia (`""`) é validado?** — *Resolver via código.*
[RESPONDIDA] Não. A RPC `create_lancamento_comissao_v2` não valida vazio; a API só checa presença (`!descricao`), que passa para `""`. Validação fraca — decidir se string vazia deve ser rejeitada.
14. **Como os filtros se comportam com período anual (`'2025'`) mencionado nos tipos TypeScript?**
— *Resolver via código.* [RESPONDIDA] Suporte anual **não implementado**. O banco armazena TEXT sem validação e as queries filtram por literal `YYYY-MM`; um `'2025'` inserido não é encontrado por filtros mensais. Apenas mensal funciona. Ver Regra 14.

Novas dúvidas levantadas pela verificação (sem resposta definitiva)

- **Recálculo em cascata ao mover comissão de período:** ao editar o `periodo` de uma comissão (Dúvida 11), os saldos de origem e destino não são reajustados. Confirmar com o cliente se isso deve gerar recálculo automático. (*cliente*)
- **Naturezas além de “Bonificação” que não deveriam comissionar:** só há check explícito para `'Bonificação'`. Naturezas como `'Devolução'` / `'Cancelamento'`, se existirem, não são tratadas. Precisa da lista válida de `natureza_operacao`. (*cliente*)
- **Campo `efetivada` sempre `true`** (migration 143): nunca é `false` nem consultado. Confirmar se é legado antes de remover. (*código/cliente*)
- **`fechar_periodo_comissao_v2` chamado duas vezes** para o mesmo (`vendedor, periodo`): o `ON CONFLICT DO UPDATE` recalcula `saldo_final` com o novo `total_pagos`, causando volatilidade. Confirmar se é intencional. (*código/cliente*)
- **Escopo RLS de `vendedor_comissao`:** a migration 143 não especifica a policy de leitura por vendedor. Verificar se existe policy separada isolando cada vendedor às suas comissões (base da Regra 22). (*código*)
- **Propósito de `oc_cliente` / `cliente_nome` desnormalizados** convivendo com a FK para `pedido_venda`: presume-se auditoria (cliente pode ser deletado), mas não há documentação. (*cliente*)

Logística & Rastreio SSW

Estado atual do módulo (verificado no HEAD de `main`, 2026-07-03)

O módulo de logística está **parcialmente desmontado / quebrado em runtime**. As 5 edge functions ainda presentes (`frete-logistica-v1`, `ssw-tracking-v1`, `origem-frete-v1`, `regiao-destino-v1`, `transportador-logistica-v1`) importam 3 módulos compartilhados que **não existem** no branch: - `supabase/functions/_shared/frete-logistica-helpers.ts` (ausente) - `supabase/functions/_shared/ssw-client.ts` (ausente) - `supabase/functions/_shared/log-crm-feature-flag.ts` (ausente)

Essas funções **falham na importação** ao subir (o único commit que teve esses arquivos foi `3ee0206`, que não está no HEAD atual). Além disso: - **Não existe** função `ssw-sweep-v1` nem cron de sweep no branch. - **Não existe** função `romaneio-logistica-v1` nem a tabela `romaneio_expedicao` no `schema_baseline.sql`. - O enum de status (`status_entrega_frete`) **existe** e é a fonte da verdade dos status: `Em Separação`, `Aguardando Coleta`, `Em Trânsito`, `Em Trânsito – Reentrega`, `Entregue`, `Agendado`, `Recusado`, `Devolvido – Trânsito`, `Devolvido – Entregue`. Não existe `Devolvido` puro nem `Aguardando Agendamento`.

Consequência: as regras marcadas **[DEPRECATED — implementação removida]** descrevem comportamento que dependia dos módulos deletados e **não está ativo hoje**. Elas ficam registradas para orientar uma eventual restauração (`git restore` a partir de `3ee0206` / commit funcional

equivalente) ou confirmação de que a remoção foi intencional.

Ação pendente de decisão do cliente: a deleção foi intencional (feature desligada) ou é resultado de merge/reset acidental? Enquanto não resolvido, este documento reflete o repositório como está, não como já esteve.

Regras de negócio

1. **[DEPRECATED — implementação removida] Entrega é definitiva e não pode ser revertida.** Uma vez que um frete atinge status `Entregue` ou `Devolvido - Entregue`, eventos posteriores (anexação de comprovantes, eventos administrativos) não devem revertê-lo para `Em Trânsito`. *Por quê:* garantir que o cliente não veja o status mudar de “entregue” de volta para “em trânsito”, o que causaria confusão e desconfiança no rastreo. *Regressão:* um frete entregue/devolvido voltando a exibir `Em Trânsito` após evento administrativo posterior (ex.: `ANEXADO COMPROVANTE`). *Nota:* a função `resolveFreteStatusFromTracking()` que implementava esse comportamento estava em `_shared/frete-logistica-helpers.ts`, atualmente ausente — regra inativa até restauração.
2. **[DEPRECATED — implementação removida] Status resolvido pelo histórico completo, não pelo último evento.** Se o último evento SSW é administrativo (não-sticky), busca-se para trás no histórico por um status sticky que predomina. *Por quê:* impedir que eventos administrativos/informativos interfiram na progressão legítima do frete (saída, entrega, devolução, recusa). *Regressão:* status sendo determinado apenas pelo evento mais recente, ignorando eventos anteriores relevantes. *Nota:* dependia de `resolveFreteStatusFromTracking()` (ausente) — regra inativa.
3. **Somente dois status são terminais.** Apenas `Entregue` e `Devolvido - Entregue` são terminais (não são mais consultados no SSW, salvo `force=true`). **Recusado é sticky mas NÃO é terminal** — pode continuar sendo consultado. *Por quê:* economizar chamadas à API SSW (limite ~20 req/s) e não gastar recursos consultando fretes já em estado final. *Regressão:* um frete terminal continuando a ser consultado no SSW; ou `Recusado` sendo tratado como terminal (deixando de ser consultado indevidamente). *Nota:* o set `TERMINAL_STATUSES` vivia em `_shared/frete-logistica-helpers.ts` (ausente). A distinção sticky ≠ terminal continua sendo a semântica correta.
4. **[DEPRECATED — implementação removida] Sweep automático horário via cron.** Atualização periódica de rastreo a cada 1 h via cron `ssw-sweep-hourly` executando `ssw-sweep-v1`; limite de 500 fretes por sweep (300 via cron); concorrência de 8 requisições simultâneas para respeitar o rate limit SSW (~20 req/s); pulando status terminais. *Por quê:* manter o rastreo atualizado sem sobrecarregar a API SSW nem os recursos internos. *Regressão:* sweep não executando; processando mais de 500 por vez; concorrência acima de 8; ou consultando fretes já entregues. *Nota:* a função `ssw-sweep-v1` não existe no branch — não há sweep automático hoje. Rastreo hoje só é preenchido on-demand via UI (`get_with_ocorrencias`), que também depende do `ssw-client.ts` ausente.
5. **[DEPRECATED — implementação removida] Cache de rastreo de 30 minutos.** Dados SSW são cacheados por 30 min: se a ocorrência mais recente for mais nova que 30 min, o polling é pulado (exceto `force=true`). O cache é baseado em `frete_logistica_ocorrencia.created_at`. *Por quê:* reduzir carga na API SSW e acelerar respostas reutilizando dados recentes. *Regressão:* polling SSW a cada requisição sem respeitar cache; ou `force=true` não contornando o cache quando necessário. *Nota:* `isCacheStale()` vivia em `_shared/ssw-client.ts` (ausente) — regra inativa.
6. **[DEPRECATED — implementação removida] Mapeamento estrito de ocorrência SSW → status.** Regras (case-insensitive): `Entrega + 01 → Entregue`; `Cliente + 02 → Agendado`; `AGENDAD* + (08) → Agendado`; `RECUSAD* → Recusado`; `DEVOLU* → Devolvido (variantes)`; `REENTREGA → Em Trânsito - Reentrega`; default → `Em Trânsito`. **DEVOLU* é checado ANTES de RECUSAD***. *Por quê:* padronizar a interpretação de eventos SSW; evitar ambiguidade quando um evento casa com múltiplos padrões. *Regressão:* código mapeado para status incorreto; inversão da

precedência de regex (ex.: `RECUSADO` antes de `DEVOLUÇÃO`); ou novos códigos sem mapeamento.

Nota: `map0correnciaToStatus()` vivia em `_shared/ssw-client.ts` (ausente) — regra inativa. A ordem `DEVOLU*` antes de `RECUSAD*` é intencional (ver Dúvidas [RESPONDIDA]).

7. **Status inicial é sempre Em Separação**. Todo frete novo criado via `frete-logistica-v1` (ou auto-create do Tiny) nasce em `Em Separação`. *Por quê:* garantir ciclo de vida consistente: começa em separação, passa por trânsito e termina em entrega/devolução/recusa. Confirmado no schema: `status_entrega ... DEFAULT 'Em Separação' NOT NULL`. *Regressão:* frete novo nascendo em `Entregue` direto (pulando estágios); ou default diferente de `Em Separação`. *Nota:* a ressalva sobre um endpoint público (`entrega-publica-v1`) criar fretes já terminais **não se aplica** — essa função não existe no branch.
8. **Idempotência por chave de acesso NFe (44 dígitos)**. A `nfe_chave_acesso` é única entre fretes ativos, garantida pelo índice parcial `uq_frete_logistica_chave_acesso ... WHERE (nfe_chave_acesso IS NOT NULL AND deleted_at IS NULL)` e pelo check `nfe_chave_44` (regex `^[0-9]{44}$`). O auto-create do Tiny trata o erro `23505` como sucesso idempotente. *Por quê:* não criar múltiplos fretes para a mesma nota; reintentos de auto-create não duplicam. *Regressão:* múltiplos fretes com a mesma chave NFe; ou auto-create falhando com erro Postgres em vez de tratar como skip idempotente.
9. **Idempotência por (empresa_id, nfe_numero)**. A combinação é única entre fretes ativos via índice parcial `uq_frete_logistica_empresa_nfe ... WHERE (nfe_numero IS NOT NULL AND deleted_at IS NULL)`. *Por quê:* mesmo com múltiplos envios do mesmo pedido do Tiny, não criar fretes duplicados. *Regressão:* múltiplos fretes com o mesmo `(empresa, nfe_numero)`; ou segundo webhook falhando ao setar o mesmo par. *Nota relacionada:* existe também `uq_frete_logistica_pedido_venda` (único por `pedido_venda_id` ativo) — terceira trava de idempotência não coberta pelas regras originais.
10. **Soft-delete em fretes, transportadores e faturas**. `frete_logistica`, transportador e fatura usam soft-delete (`deleted_at`); as queries filtram `deleted_at IS NULL`. `origem_frete` NÃO tem soft-delete (usa flag `ativo`); `regiao_destino` não tem handler de delete. *Por quê:* preservar histórico dos fretes principais mas simplificar as tabelas auxiliares. *Regressão:* hard-delete de `frete_logistica` sem soft-delete; ou query esquecendo `deleted_at IS NULL` e retornando fretes deletados. *Correção vs. contrato original:* a tabela `frete_logistica_ocorrencia` **POSSUI** coluna `deleted_at` no `schema_baseline.sql` (portanto é capaz de soft-delete e cai em CASCADE do frete pai). A afirmação de que ocorrência “usa hard-delete” não se sustenta pelo schema atual; a rotina de DELETE de ocorrências vivia no código deletado e não pode ser confirmada hoje (ver Dúvidas).
11. **[DEPRECATED — implementação removida] Dias em trânsito**. Calculado (`data_saida` até hoje) apenas em GET de lista (`list`, `list_by_status`, `get_with_ocorrencias`); retorna `null` se o frete já foi entregue ou se `data_saida` está ausente/inválida. GET por ID, POST e PUT não incluem `dias_transito`. *Por quê:* mostrar há quanto tempo o frete está na estrada (KPI); não faz sentido para fretes já entregues. *Regressão:* `dias_transito` sendo exibido para fretes entregues; ou calculado em GET por ID. *Nota:* `diasEmTransito()` vivia em `_shared/frete-logistica-helpers.ts` (ausente) — regra inativa.
12. **[DEPRECATED — implementação removida] Buckets do Dashboard/Kanban**. 5 buckets: `Em Separação`, `Aguardando Coleta`, `Em Trânsito` (consolidando `Em Trânsito` + `Em Trânsito - Reentrega`), `Entregue`, `Devolvido` (consolidando `Devolvido - Trânsito` + `Devolvido - Entregue`, já que não existe status `Devolvido` puro). Limite de 20 fretes por bucket; ordenação `data_emissao DESC, id DESC`. *Por quê:* visualizar fretes por estágio sem sobrecarregar a tela; mais recentes primeiro. *Regressão:* status novo do enum sem bucket (invisível no Kanban) — hoje `Agendado`, `Recusado` e as variantes de devolução precisam de destino explícito. *Nota:*

`DASHBOARD_BUCKETS` vivia em `_shared/frete-logistica-helpers.ts` (ausente). Há divergência não resolvida entre a estrutura de 5 buckets acima e uma variante histórica de outro branch (`Em Trânsito` , `Reentrega` , `Agendados` , `Devoluções em Trânsito` , `Recusadas`) — ver Dúvidas.

13. **[DEPRECATED — implementação removida] Feature flag mestre `FEATURE_LOG_CRM`** . Todas as edge functions de logística (`frete-logistica-v1` , `ssw-tracking-v1` , `origem-frete-v1` , `regiao-destino-v1` , `transportador-logistica-v1`) fazem gate em `FEATURE_LOG_CRM` e retornam `503` quando desabilitado. A flag deve ser checada com valor exato `'true'` (case-sensitive, sem trim). *Por quê*: ativar/desativar o módulo sem redeploy; proteger funções em desenvolvimento. *Regressão*: função de logística respondendo com a flag desabilitada; ou checagem frouxa (`TRUE` , `' true '`). *Nota*: `isLogCrmEnabledFromEnv()` vivia em `_shared/log-crm-feature-flag.ts` (ausente) — o gate hoje **quebra na importação** antes mesmo de ser avaliado. A ressalva original sobre `romaneio-logistica-v1` não fazer gate é **vazia**: essa função não existe.
14. **[DEPRECATED — implementação removida] Sub-flag `FEATURE_LOG_CRM_SSW`** . As funções que falam com o SSW (`ssw-tracking-v1` e, quando existir, o sweep) exigem **ambas** `FEATURE_LOG_CRM` e `FEATURE_LOG_CRM_SSW` em `'true'` . *Por quê*: controle granular — desabilitar só o SSW sem desligar toda a logística. *Regressão*: SSW rodando com `FEATURE_LOG_CRM_SSW` desabilitado; ou `frete-logistica-v1` passando a exigir a sub-flag (não deveria). *Nota*: `isLogCrmSswEnabledFromEnv()` vivia em `_shared/log-crm-feature-flag.ts` (ausente) — regra inativa.

Regras removidas do contrato original (refutadas pela verificação): - **LOG-008 (Romaneio agrupa notas em separação)** e **LOG-017 (validação na criação do romaneio)**: removidas. A tabela `romaneio_expedicao` **não existe** no `schema_baseline.sql` e não há função `romaneio-logistica-v1` no branch. Não há regra de negócio de romaneio ativa a documentar. Se o romaneio for reintroduzido, restabelecer estas regras. - **LOG-009 (conjunto sticky Entregue/Devolvido - Entregue/Devolvido - Trânsito/Recusado)**: absorvida nas regras 1–3 e marcada DEPRECATED junto com elas. A distinção **sticky ≠ terminal** foi preservada na regra 3.

Dúvidas em aberto

1. **Estado do repositório — remoção intencional ou acidente de merge?** O código de logística (módulos compartilhados, sweep) foi deletado, mas o schema ainda define os enums/tabelas e as 5 funções ainda importam os módulos ausentes. Commits mencionam “V 1.69 SSW automático implementado” mas o código correspondente sumiu. *Resolver via*: **cliente** (decisão de produto) + **código** (`git restore` a partir de `3ee0206` ou do commit funcional equivalente, se a remoção foi acidental).
2. **Pipeline de rastreo em produção.** Sem `ssw-sweep-v1` e sem `ssw-client.ts` , como `frete_logistica_ocorrencia` é preenchido em produção hoje? A fonte é apenas o polling manual on-demand pela UI (que também depende do módulo ausente), ou existe outro processo (edge de prod não versionada) atualizando o rastreo? *Resolver via*: **cliente** (confirmar operação real) + **código** (inspecionar edges de prod não presentes no branch).
3. **Estrutura real dos buckets do Dashboard (regra 12).** Divergência entre a estrutura de 5 buckets (`Em Separação` , `Aguardando Coleta` , `Em Trânsito` , `Entregue` , `Devolvido`) e a variante histórica de outro branch (`Em Trânsito` , `Reentrega` , `Agendados` , `Devoluções em Trânsito` , `Recusadas`). Qual é a estrutura esperada, e onde caem `Agendado` e `Recusado` ? *Resolver via*: **cliente** (definição de produto do Kanban).
4. **Delete de ocorrência: soft ou hard?** A tabela `frete_logistica_ocorrencia` tem coluna `deleted_at` (suporta soft-delete e CASCADE do frete pai), mas o contrato original afirmava hard-delete (`DELETE`) na rotina de código. Como o código deletado não está no branch, o comportamento

efetivo não pode ser confirmado. Qual é o comportamento desejado ao remover/regravar ocorrências? *Resolver via:* **cliente** (intenção) + **código** (confirmar na rotina, se/quando restaurada).

5. **Transição de status ao criar romaneio: atômica ou best-effort?** (Aplicável apenas se o romaneio for reintroduzido.) Ao criar romaneio, a transição dos fretes para **Em Trânsito** deveria abortar a criação em caso de falha (atômica) ou apenas ser logada (best-effort)? Hoje não há como decidir porque nem a função nem a tabela existem. *Resolver via:* **cliente** (definir contrato quando o romaneio for reconstruído).
6. **ssw-tracking-v1 como ferramenta de debug.** A função é descrita como debug/teste manual e permitiria polling de fretes terminais sem respeitar a regra de status terminal. Deve ser removida, restrita por permissão, ou gateada por flag? (Hoje ela também está quebrada por importar módulos ausentes.) *Resolver via:* **cliente** (decisão de exposição/segurança).
7. **Concorrência de 8 no sweep é atingida em produção?** (Aplicável apenas se o sweep for restaurado.) Não há como verificar: **ssw-sweep-v1** foi deletada e não há métricas/logs de produção acessíveis no branch. *Resolver via:* **Playwright/observabilidade** de produção, após restauração do sweep.

Dúvidas já resolvidas (viraram regra ou foram descartadas)

- **[RESPONDIDA] Recusado é terminal?** Não. **TERMINAL_STATUSES = {Entregue, Devolvido - Entregue}**. **Recusado** é sticky mas **não** terminal — continua elegível a consulta SSW. Incorporado na **regra 3**.
- **[RESPONDIDA] Ordem DEVOLU* vs. RECUSAD* no mapeamento.** A ordem correta e intencional é **DEVOLU*** antes de **RECUSAD***. Uma ocorrência que casa ambos (ex.: “DEVOLUÇÃO RECUSADA”) mapeia para **Devolvido** por precedência. Incorporado na **regra 6**.
- **[DESCARTADA] Bucket para Aguardando Agendamento.** O status **Aguardando Agendamento** **não existe** no enum **status_entrega_frete** do schema atual — dúvida sem objeto.
- **[DESCARTADA] romaneio-logistica-v1 deve fazer gate na feature flag?** A função não existe no branch — não há inconsistência a resolver.
- **[DESCARTADA] entrega-publica-v1 pode criar frete terminal?** A função não existe no branch — sem objeto.
- **[DESCARTADA] Contrato de GET romaneio/listar_disponiveis.** Sem função nem tabela de romaneio no branch — sem objeto (ver regras removidas LOG-008/017).

Permissões & Acesso

Regras de negócio do domínio de usuários, autenticação e autorização. Enforcement de CRUD de usuários acontece nas Edge Functions ***-user-v2** e nas RPCs (**005_rpc_usuarios_v2.sql**). O gating por *permission IDs* (**clientes.visualizar** , **vendas.criar** , etc.) é usado para features/UI — **não** para o enforcement do CRUD de usuários, que depende exclusivamente do campo **tipo** .

Regras de negócio

1. **Apenas backoffice cria, atualiza e deleta usuários.** As Edge Functions **create-user-v2** , **update-user-v2** e **delete-user-v2** retornam 403 se **user.tipo !== 'backoffice'** . *Por quê:* Gerenciamento de usuários determina quem acessa o sistema; permitir vendedores quebra a integridade da autenticação/autorização. *Regressão:* Um vendedor recebe 200 (em vez de 403) ao chamar create/update/delete-user-v2.

2. **Apenas backoffice altera o campo `permissoes` de qualquer usuário.** *Por quê:* Permissões controlam acesso a funcionalidades; permitir vendedores escalarem as próprias permissões ou concederem a outros quebra o modelo de autorização. *Regressão:* Um vendedor chama `update-user-v2` com `permissoes` no body e a mudança é aplicada.
3. **O campo `tipo` só pode ser `'backoffice'` ou `'vendedor'`.** Validado nas Edge Functions (whitelist de dois valores) e na RPC. *Por quê:* Toda a lógica de autorização depende de apenas dois tipos; novos tipos contornariam checagens de permissão. *Regressão:* Um usuário é criado/atualizado com `tipo = 'admin'`, `'manager'` ou outro valor.
4. **Email é único entre usuários ativos, mas múltiplos soft-deleted podem compartilhá-lo.** *Por quê:* Evita colisão de identidades ativas; soft-deleted com mesmo email suportam reativação preservando histórico. *Regressão:* Dois usuários ativos com o mesmo email, ou impossibilidade de reativar soft-deleted cujo email foi copiado de um ativo.
5. **O campo `nome` deve ter no mínimo 2 caracteres.** *Por quê:* Previne nomes incompletos que prejudicam identificação e auditoria. *Regressão:* Usuário criado/atualizado com `nome = 'a'` ou vazio.
6. **`permissoes` é sempre armazenado como array JSON (nunca null, nunca objeto).** *Por quê:* Simplifica iteração sobre permissões no back e no front; garante tipo consistente. *Regressão:* `permissoes` gravado como `{}`, `null` ou string.
7. **Vendedor só visualiza o próprio perfil.** `get-user-v2` só retorna dados se o ID solicitado for o do próprio vendedor; `list-users-v2` (RPC `list_users_v2`) retorna apenas o próprio perfil quando `tipo='vendedor'`. *Por quê:* Dados pessoais (email, admissão, dados bancários) de outros não devem ser expostos a vendedores; isolamento aumenta segurança. *Regressão:* Vendedor chama `get-user-v2` com ID de outro e recebe dados, ou `list-users-v2` retorna mais que o próprio perfil.
8. **Vendedor só atualiza o próprio perfil e nunca muda `tipo` ou `ativo`.** *Por quê:* Impede que vendedores se promovam a backoffice ou se desativem; protege o RBAC. *Regressão:* Vendedor atualiza outro usuário, ou muda `tipo` / `ativo` no próprio perfil e a mudança persiste.
9. **Exclusão é soft delete (`UPDATE ativo=false, deleted_at=NOW()`), nunca hard delete.** *Por quê:* Preserva histórico, auditoria e integridade referencial de clientes e comissões atribuídos ao usuário; hard delete quebraria FK e perderia dados. *Regressão:* Usuário deletado e registros relacionados desaparecem, ou impossibilidade de reativar usuário com soft delete.
10. **Criar usuário sem `auth_user_id` fornecido dispara convite por email (`inviteUserByEmail`), e o `auth_user_id` retornado é salvo.** *Por quê:* O usuário precisa da convocação para se autenticar no Supabase Auth. *Regressão:* Usuário criado sem `auth_user_id` não recebe convite, ou o `auth_user_id` não é registrado no banco.
11. **Criar usuário com email de um soft-deleted existente reativa aquela linha (`UPDATE deleted_at=NULL`) em vez de falhar.** Se `permissoes` não for fornecido no request, o valor antigo é preservado (o `UPDATE` de reativação só toca `permissoes` quando o campo vem no body — ver `create-user-v2` l.383). *Por quê:* Permite reutilizar emails de deletados preservando `user_id` e FKs (ponte entre criação e reativação). *Regressão:* Criar com email de soft-deleted resulta em 400 “email já cadastrado”, ou reativar perde o `user_id` original.
12. **Todas as Edge Functions (create/update/delete/list/get-user-v2) exigem JWT válido e usuário ativo (`ativo=true` e `deleted_at IS NULL`) antes de processar.** *Por quê:* Só autenticados, ativos e não deletados devem acessar os endpoints; caso contrário, um token expirado ou usuário inativo executaria ações. *Regressão:* Usuário inativo/deletado chama um endpoint e a requisição é processada (200 em vez de 401/403).

13. **Novos usuários backoffice recebem todas as permissões por padrão** (`clientes.*`, `vendas.*`, `relatorios.*`, `config.*`, `usuarios.*`, `contacorrente.*`, `produtos.*`, `comissoes.*`, `configuracoes.*`). *Por quê*: Backoffice é papel administrativo que precisa de acesso total; reduz fricção operacional. *Regressão*: Novo backoffice criado com `permissoes = []` sem acesso a nada.
14. **Novos usuários vendedor recebem um subconjunto padrão** (`clientes.visualizar/criar/editar`, `vendas.visualizar/criar/editar`, `produtos.visualizar`, `comissoes.visualizar`, `relatorios.visualizar`, `contacorrente.visualizar/criar`). *Por quê*: Vendedores trabalham com clientes, vendas e comissões, mas não devem ter acesso por padrão a configurações globais ou dados de todos os vendedores. *Regressão*: Novo vendedor recebe todas as permissões de backoffice, ou nenhuma.
15. **Permission IDs de string inválida são deduplicados e (para vendedor) validados contra whitelist**. Duplicatas são removidas via `Array.from(new Set(...))`. **⚠ VIOLAÇÃO CONHECIDA (verificação)**: a whitelist só é aplicada a **vendedor** (`sanitizeAndValidatePermissionIds` → `SUPPORTED_SELLER_PERMISSION_IDS`). Para **backoffice** (create-user-v2 l.346) as permissões passam apenas por `filter(typeof === 'string') + sanitizeInput`, **sem** checagem contra whitelist — um backoffice consegue salvar IDs inventados (`['fake.id', 'admin.totalpower']`). *Por quê (regra desejada)*: Prevenir que o cliente injete IDs inexistentes, mantendo consistência entre banco e código; a whitelist deveria valer para **ambos** os tipos. *Regressão*: `permissoes = ['fake.permission']` (especialmente com `tipo='backoffice'`) é salvo no banco.
16. **Permission IDs são sanitizados antes de comparar contra a whitelist** (trim, remove `<>`, `javascript:`, `on*=`). *Por quê*: Evita que `'<clientes.visualizar>'` passe na whitelist (já que difere de `'clientes.visualizar'`). *Regressão*: ID com caracteres especiais (`'<clientes.visualizar>'`) passa na validação e é salvo.
17. **Inputs de string (email, nome, user_login) são sanitizados** (remove `<>`, `javascript:`, `on*=`) antes de salvar. *Por quê*: Defesa em profundidade contra XSS/injection, além da constraint de JSONB. *Regressão*: Usuário criado com `nome = '<script>alert(1)</script>'` e a tag armazenada literalmente.
18. **A permissão de criar/atualizar/deletar é validada (deve ser backoffice), mas a autoria da auditoria NÃO é persistida no banco**. As RPCs `create_user_v2(p_created_by)`, `update_user_v2(p_updated_by)` e `delete_user_v2(p_deleted_by)` **aceitam e validam** o parâmetro (verificam que o ator é backoffice ativo / tipo correto), mas **não gravam** `created_by / updated_by / deleted_by` em coluna alguma — o parâmetro é usado só para validação e depois descartado. **⚠ Correção da verificação**: a versão anterior desta regra afirmava que a autoria era registrada; isso é **falso**. Não existem colunas de autoria na tabela `user`. *Por quê (o que de fato existe)*: Garantir que só backoffice execute a operação. *Regressão*: Um vendedor consegue criar/atualizar/deletar (falha de validação). Ver também Dúvida 4 (existe tabela de auditoria separada?).
19. **⚠ (VIOLAÇÃO / regra ausente) Reativação de soft-deleted pode mudar o tipo sem checagem específica**. Em create-user-v2 (l.380) o UPDATE de reativação faz `tipo: sanitizedData.tipo`, aplicando o `tipo` vindo do body sem preservar o original nem revalidar. Como só backoffice chega a esse caminho, o impacto é limitado, mas permite reativar um vendedor soft-deletado **como backoffice** — escalção de privilégio via reativação. *Por quê (regra desejada)*: Reativação deveria preservar o `tipo` original, ou a mudança de tipo deveria exigir validação explícita de backoffice. *Regressão*: Reativar soft-deleted vendedor passando `tipo='backoffice'` e a promoção ser aplicada.

Dúvidas em aberto

1. **auth_user_id na reativação**. Na reativação (soft-deleted com mesmo email), `resetPasswordForEmail(email)` é chamado por **email**, não por `user_id`. Se a conta do Supabase Auth foi hard-deletada, o reset falha (erro capturado) e o `user_id` antigo no DB não é

sincronizado. O comportamento depende da configuração soft/hard delete do Supabase Auth. →

Resolver via cliente: confirmar a expectativa de reativação quando a conta Auth original não existe mais.

2. `ultimo_acesso` só é atualizado no `get-user-v2`. Apenas `get-user-v2` (l.60-63) faz um UPDATE best-effort em `ultimo_acesso` quando o usuário lê o próprio perfil; `update-user-v2` e `list-users-v2` não. É intencional (tracking leve de atividade) ou deveria viver em outro lugar? →

Resolver via cliente: confirmar semântica esperada de `ultimo_acesso`.

3. **Backoffice sem `usuarios.criar` mesmo assim cria usuários.** [RESPONDIDA] Sim, é o comportamento atual e intencional: o enforcement de CRUD de usuários usa apenas `tipo === 'backoffice'`, nunca permission IDs. As permissões `usuarios.*` servem só para gating de UI/features. Um backoffice com `usuarios.criar` revogado ainda cria usuários. → **Resolvida via código** (create/update/delete-user-v2 checam só `tipo`).

4. **Existe tabela de auditoria separada?** A autoria (`p_created_by` / `p_updated_by` / `p_deleted_by`) é validada mas não gravada em colunas na tabela `user` (ver Regra 18). É preciso confirmar se há uma tabela de audit log separada que capture quem fez cada operação, ou se a autoria é de fato perdida. → **Resolver via cliente/código:** verificar existência de `audit_log` ou tabela equivalente.

5. **`update_dados_vendedor_v2` sem validação prévia na Edge Function.** `update-user-v2` (l.357) chama a RPC `update_dados_vendedor_v2` sem check de `tipo` /permissão na própria Edge Function; a RPC não está em `005_rpc_usuarios_v2.sql`. Se um vendedor conseguir chamá-la para outro vendedor, é bug. → **Resolver via código:** localizar a migration da RPC e confirmar se ela valida `p_user_id = p_updated_by` quando o ator é vendedor.

6. **`first_login` é retornado mas nunca inicializado no CRUD.** [PARCIAL] As RPCs retornam `u.first_login`, mas nenhuma função create/update/delete o seta ou reseta; fica com valor padrão (NULL/false). Presume-se gerenciado em outro lugar (trigger ou cliente/Auth hook). → **Resolver via código:** localizar onde `first_login` é setado (trigger no banco ou lógica de login no frontend).

7. **Enforcement de visibilidade de dados (clientes/vendas/comissões) por vendedor.** Fora do domínio de usuários. `list_users_v2` só aplica a regra “vendedor → próprio perfil”. A regra de “vendedor só vê seus próprios clientes/vendas” (se existir) vive nas RPCs de clientes/vendas. → **Resolver via código:** analisar as RPCs de clientes/vendas (outro domínio).

Produtos, Listas de Preço & Comissionamento

Regras de negócio do domínio de produtos, precificação por lista e comissionamento de vendedores. Base de código verificada: `supabase/migrations/082_commission_logic_update.sql`, `117_baseline_prod_functions_20260601.sql`, `schema_baseline.sql`, `SaleFormPage.tsx`, `StatusMixSettings.tsx`, `types/listaPreco.ts`, `types/condicaoPagamento.ts`.

Regras de negócio

1. **Preço do produto por cliente = preço da lista + desconto padrão do cliente.** O preço que um cliente vê para um produto é determinado pela lista de preço associada ao cliente, com o desconto padrão do cliente aplicado sobre o `valorUnitario` de cada item (via `percentualDescontoPadrao`). *Por quê:* Define a base comercial de cada transação; dois clientes com descontos diferentes veem preços diferentes do mesmo produto. *Regressão:* Se o desconto

padrão do cliente não for aplicado ao preço da lista, o cliente vê preço cheio; se o preço da lista muda, pedidos já criados continuam referindo-se ao valor gravado no momento (o snapshot é o item do pedido, não a lista viva).

2. **Faixa de desconto para comissão é intervalo `[min, max]` inclusivo em ambas as pontas.** A faixa é definida por `(desconto_minimo, desconto_maximo)` em percentual. O match no SQL usa `desconto BETWEEN desconto_minimo AND desconto_maximo` — ou seja, `min <= desconto <= max`, inclusivo dos dois lados (não `min < valor <= max`). `desconto_maximo = null` significa “sem limite superior” (ver Dúvida em aberto DUV_06/DUV_07). *Por quê:* Determina qual percentual de comissão o vendedor recebe conforme o desconto do cliente. *Regressão:* Se a inclusão das pontas mudar (ex.: passar a `>` ou `<`), um desconto que caía exatamente em `min` ou `max` deixa de casar e a comissão vira 0 silenciosamente.
3. **Uma lista de preço usa exatamente um modelo de comissão: `fixa` OU `conforme_desconto`.** O enum `TipoComissao = 'fixa' | 'conforme_desconto' (types/listaPreco.ts)` impede simultaneidade no schema. Nunca os dois ao mesmo tempo. *Por quê:* Sem exclusividade o cálculo fica ambíguo — qual percentual aplicar se houvesse ambos? *Regressão:* Se ambos coexistissem, o cálculo retornaria resultado inconsistente ou quebraria por campo faltando.
4. **Vendedor com `dados_vendedor."Comissão" = 2` (alíquota fixa) recebe percentual constante de `aliquotafixa`.** O percentual vem de `dados_vendedor.aliquotafixa (coalesce(..., 0))`, independente de desconto, lista ou cliente. Migration 082 aplica o tipo 2 sem consultar lista nem desconto. *Por quê:* Simplifica o comercial: vendedor tem percentual fixo em qualquer venda. *Regressão:* Se o sistema passar a buscar comissão na lista do cliente mesmo para tipo 2, o vendedor recebe menos que o combinado.
5. **Vendedor com `dados_vendedor."Comissão" = 1` (por lista) recebe o percentual de `listas_preco_comissionamento` casando a lista do cliente + o DESCONTO PADRÃO DO CLIENTE.** > **Correção da regra original:** o lookup usa `coalesce(c.desconto, 0)` — o campo `desconto` da tabela `cliente` (desconto padrão do cliente), **não** o desconto do pedido nem o `descontoExtra` da condição de pagamento. O match é `lista_preco_id = cliente.lista_de_preco AND cliente.desconto BETWEEN desconto_minimo AND desconto_maximo, ORDER BY desconto_minimo ASC, id ASC LIMIT 1`. Se nenhuma faixa casa, `v_percentual := 0`. *Por quê:* Alinha comissão ao desconto do cliente — vendedor ganha menos quando o cliente tem desconto maior. *Regressão:* Se o lookup falha (cliente sem lista, sem faixa correspondente, ou `desconto` NULL), a comissão vira 0 **silenciosamente** — sem erro ou aviso ao usuário.
6. **Valor da comissão = `round(valor_total_pedido * percentual / 100, 2)` — sempre 2 casas.** Arredondamento contábil a 2 casas via `round(..., 2)` em migration 082. (Nota: no `SaleFormPage` a função de UI `truncarPara2Casas()` **trunca** 2 casas para exibição — o valor gravado de comissão é o `round` do banco.) *Por quê:* Define precisão contábil; arredondar em posição errada acumula centavos em grande volume. *Regressão:* Se mudar para 3 casas ou 0, saldos divergem centavo a centavo ao longo do tempo.
7. **Pedido com `natureza_operacao = 'Bonificação'` (exato, case-sensitive, com til) não gera comissão; `status = bonificacao_sem_comissao`.** A checagem é `if v_pedido.natureza_operacao = 'Bonificação' sem LOWER()`. Variações de caixa (`bonificação`, `BONIFICACAO`, sem til) NÃO acionam a regra. *Por quê:* Bonificação é operação sem margem; nunca deve gerar comissão. *Regressão:* Se a bonificação gerar comissão, o vendedor ganha sobre venda de custo zero; qualquer variação de grafia contorna a regra.
8. **Constraint de faixa no banco é frouxa (`<=`); não há validação estrita no TypeScript.** > **Correção da regra original:** a constraint em `schema_baseline.sql` é `desconto_minimo <= desconto_maximo` (`<=`, permite `min == max`). **Não existe** validação estrita (`<`) no TypeScript — `types/listaPreco.ts` não valida faixa. Portanto faixas degeneradas como `[10, 10]` são aceitáveis pela constraint. *Por quê:* A intenção é garantir intervalo válido, mas hoje só o banco valida,

e de forma permissiva. *Regressão*: Faixas `[10,10]` são inseríveis; sem validação de aplicação, faixas invertidas `[20,15]` só são barradas pela constraint do banco (que rejeita porque `20 <= 15` é falso).

9. **Produto aparece no seletor de venda se `preco > 0 AND disponivel !== false AND ativo !== false` — filtro só no frontend.** Filtro em `SaleFormPage.tsx` (`produtosDisponiveisParaPedido`). Não é imposto na API. *Por quê*: Impede vender produto sem preço ou descontinuado. *Regressão*: Um produto marcado `disponivel=false` some de novos pedidos, mas pedidos já abertos continuam referenciando-o; a API não bloqueia, então é contornável fora da UI.
10. **Aba Mix exhibe só produtos “principais” — filtro client-side por tipo.** `StatusMixSettings.tsx` filtra no cliente, excluindo produtos ativados manualmente e (conforme a regra de produto) tipos promocionais/brindes. O filtro é client-side. *Por quê*: Foca o mix em produtos principais; promocionais e brindes são pontuais. *Regressão*: Se o filtro for removido, brindes aparecem como produtos preferidos e poluem os dados de mix.
11. **Apenas usuários `tipo = backoffice` podem criar/atualizar/deletar listas de preço; GET é liberado para autenticados.** A validação backoffice-only está na Edge Function (`listas-preco-v2`), não no cliente. O cliente valida `backoffice` em `App.tsx` / `SaleFormPage.tsx` para UI. *Por quê*: Preço é decisão estratégica; vendedor não muda lista sozinho. *Regressão*: Se a validação server-side for removida, um vendedor altera a lista e quebra a comissão de outros vendedores. (Ver DUV: a proteção real depende da Edge Function, não da UI.)
12. **Deletar lista de preço remove `produtos_listas_precos` e `listas_preco_comissao` antes do master.** Migration 117 deleta em ordem (produtos e comissao antes do registro-mestre), com CASCADE FK de fallback. **Débito conhecido**: políticas RLS `allow_all` em prod permitem DELETE direto, contornando a ordem manual (ver MEMORY: `rls-allow-all-pendente`). *Por quê*: Garante integridade referencial; sem ordem, sobram registros órfãos. *Regressão*: Se o DELETE manual for pulado e o CASCADE não cobrir, produtos e faixas órfãs consomem storage e geram consultas inconsistentes.
13. **Lista de produtos ordena por `descricao ASC` (alfabética).** Migration 117 usa `ORDER BY p.descricao` (alfabético). A ordenação por `created_at DESC` foi substituída por motivo de performance. (O `LIMIT 2000` citado na regra original **não foi confirmado** no SQL lido — ver Dúvida em aberto.) *Por quê*: Ordenação alfabética é determinística e barata; evita timeout de statement no Postgres. *Regressão*: Se a ordenação mudar para colunas não indexadas ou pesadas, a query pode voltar a estourar `statement_timeout`.
14. **Cliente tem `pedido_minimo`; a validação é APENAS frontend.** > **Correção da regra original**: o mínimo é validado só em `SaleFormPage.tsx` (filtro de condições). **A API não valida `pedido_minimo`** nem rejeita com 400 — é contornável via chamada direta. *Por quê*: Protege o vendedor de pedidos não viáveis; o cliente exige quantidade mínima. *Regressão*: Como não há hard-block na API, um pedido abaixo do mínimo pode ser criado fora da UI.
15. **Produto soft-deleted (`situacao = 'Excluído'`) mantém a linha; pedidos antigos continuam válidos por referência histórica.** Soft-delete preserva a row; queries de listagem filtram `.is('deleted_at', null)` (ou `situacao <> 'Excluído'`), então o produto some de listas futuras mas o pedido já criado continua referenciando `produto_id`. O item do pedido não guarda snapshot de foto/nome além do que já foi gravado. *Por quê*: Preserva histórico; um pedido antigo não deve quebrar porque o produto foi descontinuado. *Regressão*: Se o soft-delete virar hard-delete, pedidos antigos perdem a referência e quebram exibição/relatórios.
16. **`descontoExtra` da condição de pagamento é armazenado mas NUNCA aplicado ao preço nem à comissão.** > **Regra refutada/reescrita (originais 16 e 17 da entrada)**: `condicaoPagamento.descontoExtra` existe em `types/condicaoPagamento.ts`, porém **não é lido** em `SaleFormPage.tsx` nem somado em migration 082. O cálculo de comissão usa apenas

`cliente.desconto` . O `percentualDescontoExtra` do **pedido** (campo próprio do formulário) é subtraído do total na UI, mas isso é distinto do `descontoExtra` da condição de pagamento. *Por quê:* Documentado para transparência: o campo existe no modelo mas está inerte no fluxo atual. *Regressão:* Se algum código passar a somar o `descontoExtra` da condição ao desconto do cliente, a faixa de comissão (tipo 1) muda e o vendedor passa a ganhar mais/menos que hoje. Se a intenção era somar, isto é um bug latente (ver DUV).

17. **Denormalização de `nome_marca` , `nome_tipo_produto` , `sigla_unidade` no produto: CREATE valida, UPDATE não.** O CREATE busca e valida os nomes das tabelas de lookup; o UPDATE tenta buscar mas não valida, podendo deixar valores obsoletos se o lookup falhar. *Por quê:* Denormalização evita JOINs caros e acelera leitura. *Regressão:* Se o UPDATE não rebuscar os nomes, o produto exibe marca/tipo desatualizado após mudança na tabela de origem.

Dúvidas em aberto

Apenas as que a verificação considerou realmente ambíguas. As demais viraram regras acima.

- **DUV_06 / DUV_07 — Semântica de `desconto_maximo = null` na faixa de comissão.** `desconto_maximo` NULL semanticamente significa “sem limite superior”. Porém, no SQL, `desconto BETWEEN desconto_minimo AND NULL` avalia como NULL (falso lógico), então uma faixa com `max = NULL` nunca casaria via `BETWEEN` . Não foi localizado tratamento especial (ex.: `coalesce(desconto_maximo, 100)` ou `desconto_maximo IS NULL OR desconto <= desconto_maximo`) em migration 082. O TypeScript normaliza `null -> 100` ao enviar, o que evitaria o problema no fluxo normal, mas linhas gravadas direto no banco com `max = NULL` seriam invisíveis ao match. *Como resolver:* código (grep no `BETWEEN` /lookup de comissão) para confirmar se há normalização; confirmar com o cliente a intenção (“sem limite” vs “até 100%”).
- **DUV_08 — Por que `ORDER BY descricao` e não `created_at` /última modificação?** Confirmado que a ordenação é alfabética por performance, mas se isso é a UX desejada (usuário espera ver produtos recém-alterados no topo?) é decisão de produto. *Como resolver:* cliente.
- **DUV_09 — Estados virtuais do Mix (`ativo` / `inativo` / `sem_cadastro`).** A tabela `status_mix` não foi localizada em `schema_baseline.sql` ; o código menciona os estados mas a implementação (como `sem_cadastro` = ausência de row é diferenciado de `inativo` , e como a flag “ativado manualmente” é setada/resetada) não está clara. A regra original sobre `UNIQUE(cliente_id, produto_id)` e sobre a detecção invertida de reativação manual **não pôde ser confirmada** no schema lido. *Como resolver:* código (localizar a definição de `status_mix` e a lógica de ativação manual em `StatusMixSettings.tsx`).
- **DUV_11 — Atribuir lista inativa (`listas_preco.ativo = false`) a cliente novo.** A coluna `ativo` existe em `listas_preco` , mas não foi encontrada validação que impeça atribuir uma lista inativa a um cliente. Não está claro se é bloqueio ou apenas ocultação na UI. *Como resolver:* código (procurar validação de `ativo` no fluxo de atribuição de lista ao cliente).
- **DUV (nova) — Comportamento quando `dados_vendedor."Comissão"` é NULL.** Se o tipo de comissão do vendedor é NULL (não setado), migration 082 cai no `ELSE` e faz `RAISE EXCEPTION 'Tipo de comissão inválido'` . Não está claro se esse erro é capturado no frontend ou se o pedido salva e a geração de comissão falha silenciosamente/quebra. *Como resolver:* código (rastrear o call-site de `generate_vendedor_comissao` e tratamento de erro).
- **DUV (nova) — `LIMIT 2000` na lista de produtos.** A regra original menciona `LIMIT 2000` para evitar `statement_timeout` , mas o SQL de migration 117 mostra `ORDER BY descricao` sem `LIMIT` . Não confirmado se o LIMIT existe em outra camada (Edge Function/api.ts) ou se foi removido. *Como resolver:* código (grep `2000` / `.limit(` no caminho de listagem de produtos).

- **DUV (nova) — Foto do produto: lazy-load via IntersectionObserver.** A regra original afirma lazy-load de fotos por `IntersectionObserver` com fotos setadas `undefined` no SELECT de lista. **Não foi encontrada** evidência disso em `SaleFormPage`. Não confirmado se o lazy-load está em outro componente de listagem ou se a regra é obsoleta. *Como resolver:* código (grep `IntersectionObserver` / carregamento de foto sob demanda).

Relatórios

Domínio dos relatórios analíticos: Curva ABC (clientes e produtos), Mix de Produtos por Cliente, ROI de Clientes e Solicitado x Faturado. As regras abaixo governam quais dados entram nos cálculos, como são classificados/agregados e como são exportados.

Regras de negócio

1. **Classificação ABC pelo acumulado ANTERIOR.** A classificação de curva ABC (A/B/C), tanto para clientes quanto para produtos, é determinada pelo percentual acumulado ANTES de incluir o cliente/produto atual, não depois. Curva A: `acumulado_anterior < 80%`; Curva B: `80% <= acumulado_anterior < 95%`; Curva C: `acumulado_anterior >= 95%`. (Confirmado em `CustomerABCReportPage` linhas 283–294 e `ProductABCReportPage` linhas 192–205 — comentários “CORREÇÃO” e “Agora sim, acumular” deixam a ordem explícita.)
 - *Por quê:* Garante classificação consistente e previsível; evita que clientes/produtos próximos dos limiares (80% e 95%) oscilem de curva conforme a ordem de ordenação. Usado para priorização de estoque e estratégia de vendas.
 - *Regressão:* Trocar para o percentual acumulado POSTERIOR faria todos os itens de fronteira mudarem de curva; usuários que dependem de segmentos ABC estáveis veriam mudanças inesperadas a cada execução do relatório.
2. **Registros excluídos logicamente (soft delete) ficam fora de tudo.** Clientes, produtos e vendas com `deleted_at IS NOT NULL` são excluídos de todos os relatórios. Registros excluídos nunca aparecem nem entram nos cálculos de ABC, Mix, ROI ou Solicitado/Faturado. “Excluído” no domínio significa exatamente `deleted_at NOT NULL` (não existe flag `excluido` separada). (Confirmado: RPC `list_pedido_venda_v2` linha 75 `WHERE pv.deleted_at IS NULL`; RPC `list_clientes_v2` linha 53 `WHERE c.deleted_at IS NULL`.)
 - *Por quê:* Mantém a integridade dos dados e evita que registros históricos excluídos inflam métricas. A trilha de auditoria é preservada no banco, mas o usuário vê apenas dados ativos e relevantes.
 - *Regressão:* Remover o filtro `deleted_at` faria clientes excluídos reaparecerem nos rankings ABC e nos relatórios de ROI, inflando totais e contaminando métricas com contas obsoletas.
3. **Vendedor só enxerga as próprias vendas/clientes.** Vendedores (`tipo='vendedor'`) só veem vendas onde `vendedor_uuid = seu ID` ou onde estão atribuídos em `cliente.vendedoresatribuidos`. Usuários de backoffice veem todos os dados. O filtro é aplicado no nível do RPC. (Confirmado: `list_pedido_venda_v2` linhas 137–139 e 177–181 `(v_is_backoffice OR pv.vendedor_uuid = p_requesting_user_id)`; `list_clientes_v2` linhas 101–108 exigem `criado_por = requisitante OU requisitante = ANY(vendedoresatribuidos)` E `status_aprovacao = 'aprovado'`.)
 - *Por quê:* Evita vazamento de vendas entre equipes. Cada vendedor vê apenas sua carteira e contas atribuídas, protegendo cálculos de comissão e desempenho.
 - *Regressão:* Remover o filtro de vendedor exporia todas as vendas da empresa a qualquer usuário autenticado, quebrando o isolamento entre equipes e permitindo disputas de comissão ou vazamento de dados.

4. **Paginação automática carrega o dataset completo.** Os relatórios percorrem automaticamente todas as páginas retornadas pela API (máx. 100 itens/página) até atingir `totalPages`, concatenando todos os resultados em memória. A UI não impõe limite padrão — o usuário recebe o dataset completo do período filtrado. (Confirmado: `api.ts` linhas 1754–1764, loop `while (page <= totalPages)`.)
- *Por quê:* Garante que ABC, Mix e ROI considerem toda a população de clientes/produtos/vendas, não apenas os primeiros 100 registros, o que distorceria a análise para bases grandes (960+ clientes).
 - *Regressão:* Remover a paginação e devolver só a primeira página faria os relatórios exibirem dados incompletos; curvas ABC e métricas de ROI seriam calculadas sobre uma fração das vendas reais.
5. **Filtro “Concluídas” restringe a status de conclusão.** Quando `filters.statusVendas = 'concluidas'`, só entram vendas com status em `['Faturado', 'Pronto para envio', 'Enviado', 'Entregue', 'Não Entregue']` (utilitário `isStatusConcluido`). O padrão é `'todas'`, salvo seleção explícita de “Concluídas”. (Confirmado: `statusVendaUtils.ts` linhas 36–49; `CustomerABCReportPage` linha 233 usa o filtro; padrão `'todas'` na linha 100.)
- *Por quê:* Permite excluir vendas em andamento (‘Rascunho’, ‘Em Análise’, ‘Aprovado’) dos relatórios financeiros; garante que ROI e Mix reflitam apenas receita já faturada ou a faturar.
 - *Regressão:* Se o filtro ‘concluidas’ for ignorado, os relatórios misturariam rascunhos com notas faturadas, superestimando a receita e as métricas de ROI.
6. **Status Mix com três estados.** O status de ativação de produto (`status_mix`) para cada par (cliente, produto) tem três estados: `'ativo'`, `'inativo'` (explicitamente marcado) e `'sem_cadastro'` (estado virtual: não existe registro na tabela `status_mix`). A CHECK constraint do banco armazena apenas `'ativo'` e `'inativo'`; `'sem_cadastro'` é derivado no frontend quando não há registro. (Confirmado: `RelatorioMixCliente.tsx` linhas 267–273 — se existe registro usa ativo/inativo, senão retorna `'sem_cadastro'`.)
- *Por quê:* Rastreia o sortimento de produtos por cliente; distingue produtos explicitamente desativados dos que nunca foram adicionados ao mix daquele cliente. Sustenta os fluxos de gestão de mix.
 - *Regressão:* Remover a tabela `status_mix` eliminaria o rastreamento de ativação; todos os produtos seriam tratados de forma uniforme, quebrando estratégias de sortimento por cliente.
7. **ROI usa período fixo de 365 dias no SERVIDOR.** O relatório de ROI (`RelatorioROICliente`) sempre consulta os últimos 365 dias passando `periodo: '365'` como parâmetro ao endpoint `relatorios/roi-clientes`. O cálculo é feito no backend e retorna dados pré-calculados (interface `ClienteROI`: `roi`, `margemLucro`, `ltv`, etc.). **NÃO há filtro client-side de 365 dias** — o período é enviado ao servidor. (Correção da verificação: a regra original afirmava filtro client-side, o que é falso — `RelatorioROICliente.tsx` linhas 103–109 passam `periodo` ao servidor.)
- *Por quê:* Fornece uma linha-base de ROI consistente (1 ano de histórico), com o cálculo pesado sobre grande volume feito no backend.
 - *Regressão:* Se o período fixo de 365 dias for removido ou tornado configurável sem imposição server-side, os cálculos de ROI ficariam inconsistentes e não comparáveis entre janelas de tempo.
8. **Exportação CSV com BOM UTF-8 (com exceção conhecida).** As exportações CSV incluem o Byte Order Mark UTF-8 () no início e `charset=utf-8` no MIME type. **Exceção:** `SellerCommissionsPage` exporta SEM BOM, criando uma inconsistência. (Confirmado: BOM em `CustomerABCReportPage.tsx` linha 395 e `ProductABCReportPage.tsx` linha 268; `SellerCommissionsPage.tsx` linha 221 NÃO tem BOM.)
- *Por quê:* Garante que caracteres acentuados (São Paulo, Ação, etc.) sejam lidos corretamente no Excel e LibreOffice, sem corrupção ou diálogos de codificação.
 - *Regressão:* Remover o BOM causaria corrupção de acentos no Excel em muitas máquinas Windows (‘São Paulo’ vira lixo); afeta a legibilidade de nomes de clientes e descrições de produtos.

9. **Vendas trazem itens de produto ao expandir o pedido.** No GET por ID, o RPC `get_pedido_venda_v2` sempre expande o pedido com o array de produtos (`itens`), independentemente de parâmetros (linhas 324–345). Componentes que precisam do detalhe de itens (`ProductABCReportPage` , `SolicitadoFaturadoReportPage` , `RelatorioMixCliente`) acessam `venda.itens` já presente na resposta. (O parâmetro `include_itens` existe no contrato de LIST da API, mas estes componentes confiam nos itens já retornados, não o acionam explicitamente.)
- *Por quê:* Permite contratos flexíveis: o GET por ID sempre expande por conveniência, enquanto o LIST pode reduzir payload omitindo itens quando não são necessários.
 - *Regressão:* Alterar o GET por ID para não expandir itens quebraria `ProductABCReportPage` , `SolicitadoFaturadoReportPage` e `RelatorioMixCliente` , que dependem de `venda.itens` .
10. **Agrupamento opcional por dimensão de negócio.** Relatórios (CustomerABC, SolicitadoFaturado) suportam o parâmetro opcional `groupBy` : `'none'` (lista plana), `'grupo'` (grupo/rede de clientes), `'vendedor'` (vendedor) ou `'natureza'` (natureza de operação). Padrão é `'none'` . (Confirmado: `CustomerABCReportPage.tsx` linhas 20, 99, 301–337.)
- *Por quê:* Permite agregar os dados por dimensão de negócio — ex.: ABC por vendedor para identificar destaques ou baixo desempenho por equipe.
 - *Regressão:* Remover `groupBy` impediria a agregação de curvas ABC pela estrutura organizacional, limitando a profundidade analítica.
11. **Visibilidade de cliente segue o status de aprovação.** Clientes visíveis nos relatórios refletem seu `status_aprovacao` : `'aprovado'` (visível a vendedor e backoffice), `'pendente'` (apenas o vendedor criador) e `'rejeitado'` (oculto das listas normais). Os relatórios herdaram essa visibilidade via `list_clientes_v2` . **Observação:** para vendedores, o RPC filtra estritamente a `status_aprovacao = 'aprovado'` (linha 106) — ou seja, na prática o vendedor não vê clientes 'pendente'/'rejeitado' nos relatórios, nem os que ele criou (ver Dúvida em aberto). (Confirmado: `list_clientes_v2` linhas 54, 93, 101–108.)
- *Por quê:* Impede que clientes não aprovados apareçam na análise de vendas; reforça o fluxo de triagem de clientes.
 - *Regressão:* Remover o filtro por `status_aprovacao` faria clientes rejeitados aparecerem nos relatórios, inflando métricas com contas inválidas.
12. **Cálculo de quantidade perdida (Solicitado x Faturado).** No relatório “Solicitado x Faturado”, a quantidade e o percentual perdidos são calculados como: `perdaQuantidade = quantidadeSolicitado - quantidadeFaturado` ; `perdaPercentual = (perdaQuantidade / quantidadeSolicitado) * 100` (quando `solicitado > 0`). (Campos declarados na interface de `SolicitadoFaturadoReportPage.tsx` linhas 44–45.)
- *Por quê:* Identifica faltas de fornecimento (atendimento parcial, cancelamentos); ajuda a priorizar melhorias na cadeia de suprimentos.
 - *Regressão:* Alterar a fórmula de perda deturparia as taxas de atendimento; decisões gerenciais baseadas em métricas incorretas poderiam ser tomadas.
13. **Períodos-padrão diferentes por relatório.** `CustomerABCReportPage` usa período padrão `'current_month'` (mês corrente); `SolicitadoFaturadoReportPage` usa `'365'` (último ano). `RelatorioMixCliente` usa seleção customizada (padrão 30 dias) e `RelatorioROICliente` usa 365 dias fixos. (Confirmado: `CustomerABCReportPage.tsx` linha 104 `'current_month'` ; `SolicitadoFaturadoReportPage.tsx` linha 142 `'365'` .)
- *Por quê:* A curva ABC muda mensalmente e reflete atividade recente; Solicitado/Faturado acompanha perdas operacionais no longo prazo. Mix e ROI usam janelas próprias.
 - *Regressão:* Mudar os padrões de forma imprevisível faria o usuário ver intervalos de data inesperados; comparações entre relatórios ficariam inválidas.
14. **Período flexível no relatório de Mix.** O relatório de Mix (`RelatorioMixCliente`) permite seleção de período em dois modos: `'dias'` (últimos N dias, padrão 30) ou `'especifico'` (intervalo de

datas customizado). Padrão 30 dias, sobrescrevível pelo usuário. (Confirmado:

`RelatorioMixCliente.tsx` linhas 89, 171–177.)

- *Por quê*: Permite análise flexível do mix de produtos ativos; vendedores podem revisar mix recente vs. histórico sob demanda.
 - *Regressão*: Fixar o período no código impediria comparar o mix entre janelas de tempo, limitando a análise de estratégia de produto.
15. **numeroPedidos no Mix conta ocorrências de item, não pedidos únicos.** Em `RelatorioMixCliente`, o campo `numeroPedidos` de cada produto é incrementado (`+= 1`) dentro do `venda.itens.forEach`. Assim, um produto que aparece em 3 itens distribuídos por 2 pedidos recebe `numeroPedidos = 3`, não 2. (Confirmado: `RelatorioMixCliente.tsx` linha 237 — semântica do nome é enganosa, mas o comportamento é esse no código.)
- *Por quê*: Reflete a frequência de aparição do produto entre os itens do período (comportamento atual documentado, não necessariamente o desejado).
 - *Regressão*: Alterar para contar pedidos únicos mudaria os números exibidos historicamente; qualquer relatório salvo/comparado deixaria de bater. Antes de “corrigir”, validar a intenção de negócio (ver Dúvida em aberto).
16. **ProductABC agrega por produto entre todos os pedidos.** Em `ProductABCReportPage`, produtos que aparecem em múltiplos pedidos são agregados por `produtoId` num `Map`, somando quantidade e valor. Múltiplas ocorrências viram uma única linha de produto com totais agregados — nunca há detalhe por pedido. (Confirmado: `ProductABCReportPage.tsx` linhas 145–168.)
- *Por quê*: A curva ABC de produtos precisa do total agregado por produto para ranquear corretamente por participação na receita.
 - *Regressão*: Deixar de agregar (mostrar linha por pedido) quebraria o ranking ABC de produtos, que pressupõe um item por produto.

Dúvidas em aberto

As dúvidas abaixo continuam ambíguas após a verificação (as demais foram convertidas em regras ou respondidas). Marcadas `[RESPONDIDA]` as que a análise de código já esclareceu.

1. **Filtro de vendedor no ProductABC é só client-side?** `ProductABCReportPage` carrega `api.get('vendas')` sem `include_itens` e depois filtra por cliente/vendedor na UI (linha 123). Não há evidência de enforcement backend do `vendedorId` nesse caminho. Se um vendedor usar a página, o RPC ainda retorna apenas as vendas dele (filtro `vendedor_uuid`), mas o filtro de vendedor selecionável na UI não passa ao servidor. **[RESPONDIDA parcialmente]** — o isolamento por vendedor é garantido pelo RPC (Regra 3); o filtro de vendedor da UI é apenas refino client-side sobre o conjunto já autorizado. *Resolver via*: código/Playwright (confirmar que backoffice ao selecionar um vendedor obtém apenas as vendas dele — refino client-side, o que é aceitável).
2. **Vendedor deveria ver seus clientes ‘pendente’/‘rejeitado’ nos relatórios?** `list_clientes_v2` filtra vendedores estritamente a `status_aprovacao = 'aprovado'` (linha 106), então clientes ‘pendente’ ou ‘rejeitado’ nunca aparecem para o vendedor nos relatórios — nem os que ele mesmo criou. É a regra pretendida ou uma restrição forte demais? *Resolver via*: cliente (decisão de negócio).
3. **Vendas ‘Cancelado’ devem aparecer na lista, mas fora das stats?** O RPC `list_pedido_venda_v2` exclui `status = 'Cancelado'` das estatísticas (linha 159 `AND pv.status != 'Cancelado'`), mas não exclui vendas canceladas da lista principal (linhas 136–155). Vendas canceladas aparecem na lista de pedidos, porém ficam de fora das stats. É intencional (auditoria) ou bug? *Resolver via*: cliente (decisão de negócio) + código.
4. **numeroPedidos do Mix: contar itens ou pedidos únicos?** Ver Regra 15 — hoje conta ocorrências de item, não pedidos únicos. O nome do campo sugere “número de pedidos”. Confirmar a intenção de negócio antes de alterar. *Resolver via*: cliente (decisão de negócio) + Playwright (validar o valor exibido).

5. Inconsistência de BOM na exportação de comissões é bug ou intencional?

`SellerCommissionsPage` exporta CSV sem BOM enquanto os demais relatórios incluem BOM (ver Regra 8). Não há padronização detectada. Corrigir (adicionar BOM) ou é aceitável? *Resolver via:* cliente (definir padrão de exportação).

6. Quais naturezas de operação têm `gera_receita = true`? O RPC `list_pedido_venda_v2` (linha 129) lê `no.gera_receita` da tabela `natureza_operacao`, com `COALESCE(..., FALSE)`. O mapeamento de quais naturezas geram receita (ex.: Venda = true, Devolução = false) não está documentado no código. *Resolver via:* código (consultar dados da tabela `natureza_operacao`) + cliente (validar o mapeamento de negócio).

7. Filtro de produto no Solicitado x Faturado é omissão ou por design?

`SolicitadoFaturadoReportPage` permite filtrar por cliente, vendedor e natureza, mas não expõe filtro por produto (interface de `filters` sem `produtoId`), embora calcule métricas por produto. Falta a UI para isolar um único produto. *Resolver via:* cliente (decisão de escopo) + Playwright (confirmar ausência do filtro).

Conta Corrente

Domínio que registra **compromissos financeiros** com clientes (investimentos e ressarcimentos), os **pagamentos** que os quitam, as **categorias** de classificação e as **formas de pagamento**. O saldo/estado é sempre derivado dos pagamentos, nunca armazenado.

Fontes principais: - RPC compromissos/pagamentos:

`supabase/migrations/067_rpc_conta_corrente_v2.sql` - RPC categorias:

`supabase/migrations/022_rpc_categorias_conta_corrente_v2.sql` - RLS:

`supabase/migrations/068_rls_conta_corrente.sql` - Migrations de `categoria_id` /

`categoria_nome`: `072_*`, `074_*` - Handlers: `supabase/functions/conta-corrente-v2/index.ts`, `categorias-conta-corrente-v2/index.ts`, `formas-pagamento-v2/index.ts`

Regras de negócio

- Tipo do compromisso deve ser `investimento` ou `ressarcimento`.** Na criação, `tipo_compromisso` é validado como obrigatório e restrito a esses dois valores (case-insensitive): `IF p_tipo_compromisso IS NULL OR LOWER(p_tipo_compromisso) NOT IN ('investimento', 'ressarcimento') THEN RAISE EXCEPTION` (067, ~linha 440). *Por quê:* segregar tipos de obrigação financeira (investimento em produto/serviço vs. ressarcimento de valor devido) para análise e relatórios por tipo. *Regressão:* aceitar tipo diferente (ex. “Empréstimo”) ou NULL deixaria os dados inconsistentes e os relatórios por tipo ambíguos. *Ressalva:* a validação existe apenas na RPC. A tabela `conta_corrente_cliente` não é criada neste repositório e **não há constraint `CHECK / NOT NULL conhecido`** no schema; inserção direta via RLS (`WITH CHECK (true)`) contorna a regra (ver CCR-013 abaixo).
- Valor do compromisso deve ser maior que zero na criação.** `IF p_valor IS NULL OR p_valor <= 0 THEN RAISE EXCEPTION 'valor deve ser maior que zero'` (067, ~linha 433). *Por quê:* evitar compromissos sem valor ou negativos, que invalidariam saldo e cálculo de pagamentos. *Regressão:* compromisso com valor 0/negativo/NULL torna o cálculo de pendência inválido. *Ressalva (bug conhecido):* no UPDATE, o handler faz `body.valor ? Number(body.valor) : null — valor = 0` vira `null` silenciosamente e a RPC de update aceita NULL. Ver Dúvidas em aberto (DUV-001).
- Título do compromisso é obrigatório e deve ter no mínimo 2 caracteres.** `IF p_titulo IS NULL OR LENGTH(TRIM(p_titulo)) < 2 THEN RAISE EXCEPTION` (067, ~linha 437). *Por quê:* garantir identificação clara do compromisso para auditoria, relatórios e comunicação com o cliente. *Regressão:*

título vazio/1 caractere/NULL impede identificar o que é o compromisso.

4. **Data do compromisso é obrigatória.** `IF p_data IS NULL THEN RAISE EXCEPTION 'data é obrigatória'` (067, ~linha 429). *Por quê:* estabelecer data de criação/vencimento para controlar o fluxo temporal de obrigações e pagamentos. *Regressão:* sem data é impossível ordenar/filtrar por período; relatórios ficam inválidos.
5. **Status do compromisso é calculado dinamicamente, nunca armazenado.** A cada consulta: soma dos pagamentos = 0 → **Pendente**; soma ≥ valor → **Pago Integralmente**; caso contrário (1–99%) → **Pago Parcialmente**. `CASE WHEN COALESCE(SUM(pac.valor_pago),0) = 0 THEN 'Pendente' WHEN ... >= ccc.valor THEN 'Pago Integralmente' ELSE 'Pago Parcialmente' END` (067, ~linhas 113–117). *Por quê:* refletir em tempo real o estado do pagamento sem coluna duplicada sujeita a dessincronização. *Regressão:* status persistido sem sincronização mostraria estado desatualizado após pagamento. *Nota:* como é derivado, **deletar um pagamento recalcula o status na próxima consulta** automaticamente (ver DUV-006, RESPONDIDA). O valor `Cancelado` existe no type `StatusCompromisso` mas **nunca é gerado** por nenhuma RPC/fluxo (ver DUV-005).
6. **A soma dos pagamentos de um compromisso não pode exceder o valor total.** `IF (v_valor_total_pago + p_valor_pago) > v_valor_compromisso THEN RAISE EXCEPTION 'Valor total pago não pode exceder o valor do compromisso'` (067, ~linhas 884–886). *Por quê:* evitar overpayment e manter o equilíbrio financeiro. *Regressão:* soma > valor geraria crédito indevido ou fatura errada. *Comportamento* (DUV-008, RESPONDIDA): retorna HTTP **400** com a mensagem acima (handler mapeia keywords de validação para 400).
7. **Categoria é opcional em compromissos e pagamentos.** A RPC de criação **não valida categoria obrigatória**; o handler passa `p_categoria_id` podendo ser `null`. Não há FK obrigatória conhecida em `conta_corrente_cliente`. *Por quê:* permitir categorização flexível (infraestrutura, garantia, etc.) sem forçar o preenchimento. *Regressão:* tornar categoria obrigatória bloquearia fluxos legítimos sem categorização. *Ressalva:* a verificação apontou incerteza sobre o tipo real de `categoria_id` na tabela (migrations 072/074 tratam como UUID no SELECT, handler manipula como string trimada). Ver Dúvidas em aberto.
8. **Nome de categoria: mínimo 2 caracteres e único case-insensitive entre categorias ativas.** `IF LENGTH(TRIM(p_nome)) < 2 THEN RAISE EXCEPTION` e `IF EXISTS (... WHERE LOWER(TRIM(c.nome)) = LOWER(TRIM(p_nome)) AND c.deleted_at IS NULL) THEN RAISE EXCEPTION` (022, ~linhas 33 e 37–42). *Por quê:* evitar categorias vazias/genéricas e prevenir duplicatas que confundem relatórios. *Regressão:* duas categorias com mesmo nome (ou nome curto) tornam a análise por categoria ambígua.
9. **Forma de pagamento é obrigatória em pagamentos — via nome (`p_forma_pagamento`), não por ID.** A RPC valida `IF p_forma_pagamento IS NULL OR LENGTH(TRIM(p_forma_pagamento)) < 2 THEN RAISE EXCEPTION` (067, ~linha 836). O handler exige que ao menos um dos campos (`formaPagamento` / `forma_pagamento` / `formaPagamentoId` / `forma_pagamento_id`) esteja presente (~linhas 247–248). *Por quê:* rastrear como foi pago (dinheiro, cheque, TED, PIX...) para auditoria e conciliação. *Regressão:* pagamento sem forma impede saber como foi realizado. *Bug conhecido:* se o cliente enviar **apenas `formaPagamentoId` (numérico) sem `formaPagamento` (texto)**, o handler não busca o nome da forma antes de chamar a RPC, e `p_forma_pagamento` chega NULL — a RPC **falha** exigindo texto ≥ 2 caracteres. A regra é “obrigatória por nome”; o caminho por ID puro está quebrado.
10. **Data de pagamento é obrigatória.** `IF p_data_pagamento IS NULL THEN RAISE EXCEPTION 'data_pagamento é obrigatória'` (067, ~linha 832). *Por quê:* registrar quando o pagamento ocorreu para fluxo de caixa e vencimentos. *Regressão:* pagamento sem data invalida relatórios de fluxo temporal.

11. **Cliente vinculado ao compromisso deve existir e não estar deletado (soft delete).** `SELECT ... FROM public.cliente c WHERE c.cliente_id = p_cliente_id AND c.deleted_at IS NULL; IF NOT FOUND THEN RAISE EXCEPTION 'Cliente não encontrado' (067, ~linhas 446–454).` *Por quê:* integridade referencial — evitar compromissos órfãos de clientes inexistentes/desativados. *Regressão:* compromisso com `cliente_id` inválido/deletado quebra consultas e relatórios.
12. **Vendedor só cria/atualiza/deleta compromissos e pagamentos dos SEUS clientes; backoffice vê tudo.** Para compromissos: `IF v_user_tipo = 'vendedor' THEN IF NOT EXISTS (SELECT 1 FROM cliente_vendedores cv WHERE cv.cliente_id = p_cliente_id AND cv.vendedor_id = p_created_by) THEN RAISE EXCEPTION (067, ~linhas 470–477).` Para pagamentos há check equivalente com mensagem “Vendedor não tem permissão para criar pagamento para este compromisso” (~linhas 867–874). Backoffice ignora o check. *Por quê:* segurança — vendedor não acessa dados de clientes de outros vendedores. *Regressão:* vazamento — vendedor veria/editaria/deletaria compromissos de cliente alheio. *Nota (DUV-004, RESPONDIDA):* a validação vive só na RPC; o handler apenas repassa `p_created_by`. O erro chega ao usuário como HTTP 400 com a mensagem da RPC.
13. **Apenas backoffice cria/atualiza/deleta categorias; vendedores apenas leem (read-permissive, write-restrictive).** Handler rejeita POST/PUT/DELETE de não-backoffice: `IF user.tipo != 'backoffice' THEN throw` (categorias-conta-corrente-v2, ~linhas 168–169). GET **não** filtra por tipo — vendedores listam normalmente (~linhas 126–164; RPC `list_categorias_conta_corrente_v2` não filtra por usuário). A RPC de criação também valida backoffice (022, ~linhas 54–56). *Por quê:* centralizar a gestão de categorias e evitar duplicatas/lixo criados por vendedores. *Regressão:* vendedor com escrita fragmentaria e desorganizaria as categorias.
14. **Forma de pagamento não pode ser deletada se tiver condições de pagamento vinculadas.** `IF condicoes && condicoes.length > 0 THEN throw new Error('Não é possível excluir...')` (formas-pagamento-v2, ~linhas 283–285). *Por quê:* integridade referencial — evitar condições de pagamento órfãs. *Regressão:* deletar forma em uso deixaria condições com referência inválida.
15. **Categorias usam soft delete (`deleted_at IS NOT NULL` = deletada).** `UPDATE public.categorias_conta_corrente SET deleted_at = NOW() ...` (022, ~linhas 336–338). A unicidade de nome (CCR-008) e as listagens consideram `deleted_at IS NULL`. *Por quê:* preservar histórico e auditoria — o registro não some fisicamente. *Regressão:* delete físico perderia histórico e quebraria relatórios antigos. *Contraste importante:* **compromissos NÃO têm soft delete** — a tabela `conta_corrente_cliente` não possui coluna `deleted_at`. O handler bloqueia DELETE com “Exclusão de compromisso não está implementada. A tabela não possui campo `deleted_at`.” (conta-corrente-v2, ~linha 722), embora a RLS tecnicamente permita DELETE (ver DUV-002).

Dúvidas em aberto

- **DUV-001 — Valor 0 no UPDATE vira NULL:** No update de compromisso, `body.valor ? Number(body.valor) : null` (handler ~linha 695) converte `valor = 0` para `null`, e a RPC de update aceita NULL — contornando a validação `> 0`. **[PARCIALMENTE RESPONDIDA]** É uma brecha/bug confirmado no código: a RPC não distingue “usuário passou 0” de “usuário não passou valor”. Falta confirmar com o **cliente** se manter o valor anterior ao omitir/enviar 0 é intencional ou deve ser rejeitado. *Resolver via: cliente.*
- **DUV-002 — Delete físico de compromisso:** A RLS permite `DELETE` a `authenticated` (`WITH CHECK (true)`, 068 ~linhas 46–50), mas o handler bloqueia intencionalmente (a tabela não tem `deleted_at`). **[RESPONDIDA]** Pela aplicação, compromisso **não é deletável** hoje. Falta decisão de produto: implementar soft delete (exigiria coluna + filtros `deleted_at IS NULL` em todas as queries) ou manter bloqueio. *Resolver via: cliente.*

- **DUV-005 — Status `Cancelado`** : O type `StatusCompromisso` (contaCorrente.ts ~linha 4) inclui `Cancelado` , mas nenhuma RPC/migration/endpoint o gera. **[RESPONDIDA]** Não existe fluxo de cancelamento — o type está dessincronizado da implementação. Falta decidir com o **cliente** se cancelamento é um requisito (criar fluxo) ou se o valor deve ser removido do type. *Resolver via: cliente.*
 - **DUV-010 — RLS `WITH CHECK (true)` permite burlar as validações de negócio**: Todas as policies de INSERT/UPDATE/DELETE usam `WITH CHECK (true)` sem validação (068 ~linhas 33, 41, 50). Escrita direta na tabela (service_role/backoffice) ignora as RPCs e injeta dados inválidos (tipo fora do domínio, valor ≤ 0 , soma $>$ valor, etc.). **[RESPONDIDA — débito de segurança]** Confirmado; severidade alta. Falta alinhar com o **cliente** se as validações devem migrar para constraints/triggers na tabela (defesa em profundidade) além das RPCs. *Resolver via: cliente.*
 - **Tipo real de `categoria_id`** : Migrations 072/074 tratam `categoria_id` como UUID no SELECT, mas o handler o manipula como string trimada (~linha 625). Se a coluna for UUID, uma string malformada pode falhar silenciosamente ou virar NULL. Confirmar o tipo real da coluna em `conta_corrente_cliente` (schema não está neste repositório). *Resolver via: código/banco (inspecionar a tabela em produção).*
 - **DUV-007 — Flag `Parcelamento` desatualizado em condição de pagamento**: No UPDATE, `Parcelamento / intervaloParcela` só é recalculado se `body.prazoPagamento !== undefined` (condicoes-pagamento-v2 ~linhas 377–392). Um UPDATE parcial que muda só `descontoExtra` sem reenviar `prazoPagamento` deixa a flag desatualizada. **[RESPONDIDA — falha conhecida]** Confirmado no código. Falta decidir se o handler deve recalcular sempre a partir do estado persistido. *Resolver via: código.*
 - **DUV-009 — `intervalo_parcela` vazio em prazos inválidos**: O parser (`processarPrazoPagamento`) filtra vazios e não-numéricos; `'0/0/0'` → `[0,0,0]` (válido), mas `'////'` → `[]` (retorno `{ quantidadeParcelas: 1, prazoPagamento: 0, intervaloParcela: [] }`). **[RESPONDIDA — risco baixo]** O parser é defensivo; o risco recai em consumidores que assumem `.length > 0` (ex. `tiny-enviar-pedido`, não analisado aqui). Falta verificar o comportamento na emissão de pedido. *Resolver via: código (auditar `tiny-enviar-pedido`).*
-